บทที่ 7 SQL: ภาษานิยามข้อมูล

Data Definition Language

วัตถุประสงค์

- สามารถอธิบายความหมายและวัตถุประสงค์ของการใช้ SQL ได้
- สามารถอธิบายวิวัฒนาการของ SQL ได้
- สามารถอธิบายชนิดข้อมูลมาตรฐานที่ SQL รองรับ
- สามารถอธิบายวัตถุประสงค์ของการกำหนดเงื่อนไขบังคับบูรณภาพด้วย SQL ได้
- สามารถสร้าง แก้ไข และลบตารางข้อมูลด้วย SQL ได้
- สามารถสร้างเงื่อนไขบังคับบูรณภาพด้วย SQL ได้
- สามารถสร้าง แก้ไข และลบดัชนีด้วย SQL ได้
- สามารถสร้าง แก้ไข และลบวิวด้วย SQL ได้

คำสำคัญ: ภาษาสอบถามแบบมีโครงสร้าง (structured query language—SQL); ภาษานิยามข้อมูล (data definition language—DDL); CREATE DATABASE; CREATE TABLE; ALTER TABLE; DROP TABLE; CREATE INDEX; DROP INDEX; CREATE VIEW; DROP VIEW

บทที่ 7 SQL: ภาษานิยามข้อมูล

Data Definition Language

7.1 บทนำ

ในบทนี้เป็นหัวใจของการพัฒนาระบบจัดการฐานข้อมูลเชิงสัมพันธ์หลังจากที่ได้ออกแบบฐานข้อมูลแล้ว เป็นการอธิบายถึงการใช้เครื่องมือหลักในการจัดการระบบจัดการฐานข้อมูลเชิงสัมพันธ์ซึ่งได้แก่คำสั่ง SQL SQL เป็นภาษามาตรฐานที่ใช้กับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ในบทนี้อธิบายถึงกลุ่มของคำสั่ง SQL ที่ใช้ในการนิยามข้อมูลหรือ DDL เป็นสำคัญ โดยเริ่มจากการอธิบายถึงวัตถุประสงค์ของ SQL อย่างกระชับและอธิบายวิวัฒนาการของ SQL ในหัวข้อที่ 7.2.1 และ 7.2.2 จากนั้นเป็นการ แนะนำชนิดของข้อมูลมาตรฐานที่ใช้กับ SQL ในหัวข้อ 7.3 จากนั้นเป็นการอธิบายและแสดงตัวอย่างการใช้งาน DDL ในหัวข้อ 7.4 โดยเริ่มจากการสร้างฐานข้อมูลและตารางข้อมูลโดยใช้คำสั่ง CREATE ในหัวข้อที่ 7.4.1 และ 7.4.2 การแก้ไขโครงสร้างของตารางด้วยคำสั่ง ALTER ในหัวข้อ 7.4.3 และการลบฐานข้อมูล/ตารางข้อมูลด้วยคำสั่ง DROP ในหัวข้อที่ 7.4.4

การนิยามโครงสร้างของข้อมูลในระบบจัดการฐานข้อมูลนั้น เราจำเป็นต้องกำหนดข้อกำหนดบังคับเพื่อให้ข้อมูลมีความถูกต้องหรือมีบูรณภาพซึ่งอาศัย SQL อาศัย CONSTRAINTS ในการกำหนดเงื่อนไขบังคับต่างๆ ดังได้อธิบายไว้ในหัวข้อ 7.5 สำหรับหัวข้อที่ 7.6 เป็นการสร้างดัชนีเพื่อกำหนดรูปแบบเชิงกายภาพของฐานข้อมูลเพื่อเพิ่มประสิทธิภาพของการแสดงและจัดการข้อมูล และหัวข้อที่ 7.7 เป็นการอธิบายการสร้างและจัดการวิว

7.2 แนวคิดทั่วไปเกี่ยวกับ SQL

ระบบจัดการฐานข้อมูลเชิงสัมพันธ์อาศัยภาษามาตรฐานสำหรับดำเนินการเกี่ยวกับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ เพื่อความสะดวกและประสิทธิภาพในการดำเนินการกับฐานข้อมูล SQL (structured query language) เป็นภาษาที่ได้รับการยอมรับอย่างแพร่หลายมากที่สุดในการดำเนินกับฐานข้อมูลเชิงสัมพันธ์ ซึ่ง SQL นี้อาศัยพื้นฐานของการดำเนินการพีชคณิตเชิงสัมพันธ์และแคลคูลัสเชิงสัมพันธ์ ในการดำเนินการกับฐานข้อมูล SQL ใช้สำหรับแสดง และจัดการข้อมูล SQL แบ่งออกเป็น 2 ประเภทใหญ่ๆ ได้แก่ภาษาที่ใช้ในการนิยามข้อมูล (data definition language—DDL) เป็นกลุ่มของคำสั่ง SQL ที่ใช้ในการนิยมโครงสร้างของฐานข้อมูลต่างๆ เช่นการกำหนดโครงสร้างของฐานข้อมูล การกำหนดโครงสร้างของตารางข้อมูลต่างๆ ในบทนี้อธิบายถึงกลุ่มคำสั่ง DDL นี้ คำสั่ง SQL อีกประเภทหนึ่งได้แก่ภาษาที่ใช้ในการจัดการข้อมูล (data manipulation language—DML) เป็นกลุ่มของคำสั่งที่ใช้ในการเพิ่ม แก้ไข และลบข้อมูลใดๆ ในฐานข้อมูล ซึ่งฐานข้อมูลนั้นๆ ถูกสร้างขึ้นด้วย DDL ก่อนแล้ว ทั้งนี้ยังมีกลุ่มของคำสั่ง SQL อีกกลุ่มหนึ่งซึ่งสามารถแยกออกมาเป็นประเภทเฉพาะจาก DDL ได้แก่คำสั่งควบคุม (control language) ใช้สำหรับสร้างและกำหนดสิทธิของผู้ใช้งานฐานข้อมูล

DDL อธิบายในบทนี้ DML อธิบายไว้ในบทที่ 8 ซึ่งเป็นบทถัดไป และ control language ได้อธิบายในบทที่ 9

7.2.1 วัตถุประสงค์

SQL มีวัตถุประสงค์เพื่อใช้ในการแสดง และจัดการข้อมูลในฐานข้อมูลเชิงสัมพันธ์ เพื่อการใช้ฐานข้อมูลเชิงสัมพันธ์อย่างเป็นมาตรฐานและมีประสิทธิภาพสูงสุด SQL เป็นภาษาที่สามารถจัดการฐานข้อมูลเชิงสัมพันธ์ได้อย่างเต็มรูปแบบเป็นไปตามกฎ 12 ข้อของ E.F. Codd

7.2.2 วิวัฒนาการ (Connolly and Begg, 2004)

SQL ถือกำเนิดขึ้นพร้อมๆ กับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ซึ่งได้แก่การพัฒนาและประยุกต์แนวคิดของแบบจำลองเชิงสัมพันธ์ขึ้นมาเป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ในช่วงทศวรรษที่ 1970 ใน System R ของบริษัท IBM สำหรับ SQL ในยุคแรกๆ ที่ใช้จัดการกับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ System R นั้นมีชื่อเรียกว่า SEQUEL ย่อมาจาก Structured English Query Language สำหรับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ Ingres อีกระบบหนึ่งที่พัฒนาในช่วงไล่เลี่ยกันที่ University of California, Berkeley นั้นใช้ภาษา QUEL แต่ในท้ายที่สุดได้หันมาใช้ SQL เช่นกันในการใช้งานจริง

ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ที่เริ่มใช้ในเชิงพาณิชย์ในปี ค.ศ. 1979 ใช้ SQL เป็นภาษาที่ใช้ในการจัดการฐานข้อมูล เป็นของบริษัท Oracle ซึ่งแต่เดิมใช้ชื่อว่าบริษัทว่า Relational Software, Inc. ต่อมามีการจำหน่ายซอฟต์แวร์สำหรับจัดการฐานข้อมูลเชิงสัมพันธ์โดยใช้ SQL อย่างแพร่หลาย

เนื่องจากมีการนำเสนอผลิตภัณฑ์จากผู้ผลิตระบบจัดการฐานข้อมูลเชิงสัมพันธ์หลายราย จึงจำเป็นต้องมีการกำหนดมาตรฐานของรูปแบบของคำสั่ง SQL เพื่อความเป็นเอกภาพในการใช้งาน โดย American National Standard Institute—ANSI ได้กำหนดมาตรฐานของ SQL ขึ้นเป็นครั้งแรกในปี ค.ศ. 1986 และต่อมากลายเป็นมาตรฐานในระดับนานาชาติโดย International Organization for Standardization—ISO ในปี ค.ศ. 1987 รูปแบบของคำสั่ง SQL ในปี ค.ศ. 1986 นี้เรียกว่า SQL-86 แม้ว่าจะมีมาตรฐานกำหนดไว้เพื่อให้ผู้ผลิตระบบจัดการฐานข้อมูลแต่ละรายพัฒนาเพื่อความเข้ากันได้ ผู้ผลิตแต่ละรายได้พัฒนาส่วนเพิ่มเติมจาก SQL ในผลิตภัณฑ์ของตนเองเพื่อเพิ่มประสิทธิภาพและความสามารถในการทำงานของระบบจัดการฐานข้อมูลในสูงขึ้น ในปี ค.ศ. 1992 จึงได้มีการกำหนดรูปแบบของคำสั่ง SQL ที่เพิ่มความสามารถในการจัดการฐานข้อมูลเพิ่มขึ้นให้เป็นมาตรฐาน โดยรู้จักกันในชื่อ SQL-92 หรือ SQL2 เป็นการปรับปรุง SQL ครั้งใหญ่ โดย SQL2 ครอบคลุมการจัดการระบบฐานข้อมูลได้ครบถ้วนและสมบูรณ์กว่า SQL-86 อาจกล่าวว่า SQL2 เป็นมาตรฐานที่ต่ำที่สุดที่ระบบจัดการฐานข้อมูลปัจจุบันจะต้องรองรับ

มีการกำหนดมาตรฐานครั้งสำคัญอีกครั้งหนึ่งในปี ค.ศ. 1999 โดย SQL:1999 หรือ SQL3 ที่กำหนดขึ้นใหม่นี้เป็นการปรับปรุงครั้งสำคัญ SQL3 นั้นยังคงรองรับการทำงานหลายๆ อย่างที่ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ในปัจจุบันสามารถทำได้ ซึ่งคุณสมบัติสำคัญๆ ที่สามารถดำเนินการได้ใน SQL3 นี้ได้แก่การเพิ่มความหลากหลายของข้อคำถาม การสร้าง triggers การเขียนคำสั่ง SQL ให้อยู่ในรูปของภาษาเชิงกระบวนการและโครงสร้าง (มีคำสั่งในการเลือกทำและทำซ้ำเหมือนในภาษาคอมพิวเตอร์ระดับสูง เช่น if..else และ for ในภาษา C) และการรองรับความสามารถบางประการของฐานข้อมูลเชิงวัตถุ

นอกจากนี้ยังมีมาตรฐาน SQL:2003, SQL:2006 และ SQL:2008 ที่เพิ่มความสามารถของ SQL ไปตามยุคสมัย เช่นการใช้งาน XML (Extensible Mark-up Language) และการกำหนดมาตรฐานโครงสร้างของคำสั่งเชิงกระบวนการให้ชัดเจนและมีความสามารถมากขึ้น

7.3 ชนิดของข้อมูล

ประเภทของข้อมูลที่เราใช้งานนั้นระบบจัดการฐานข้อมูลมีความจำเป็นต้องแทนด้วยรูปแบบที่สามารถจัดการได้ด้วยระบบคอมพิวเตอร์ ระบบจัดการฐานข้อมูลของผู้ผลิตแต่ละรายจัดการข้อมูลในลักษณะต่างๆ กัน จึงมีความจำเป็นที่จะต้องกำหนดมาตรฐานของข้อมูลที่ SQL รองรับ โดยชนิดของฐานข้อมูล SQL:1999 หรือ SQL3 เป็นชนิดข้อมูลที่ระบบจัดการฐานข้อมูลส่วนใหญ่รองรับในปัจจุบันตามมาตรฐานของ ISO ซึ่งมีรายการดังต่อไปนี้

ตารางที่ 7-1 ชนิดของข้อมูลที่ SQL รองรับ

ชนิดข้อมูล	ขอบเขตของค่าที่รองรับ	คำอธิบาย
BIT	0, 1	ค่าบิต 0 และ 1 เท่านั้น ซึ่งเป็นชุดของบิตเรียงต่อกันได้
BLOB	ชุดของ 0, 1	ค่าบิต 0 และ 1 เท่านั้น ซึ่งเป็นชุดของบิตเรียงต่อกันขนาดใหญ่มาก—Binary Large Object โดยปกติเราใช้เก็บข้อมูลไฟล์ต่างๆ เช่นรูปภาพ ไฟล์เอกสาร ไฟล์วิดีโอ และไฟล์เสียง เป็นต้น
CHAR	ชุดของตัวอักษร ASCII	ตัวอักษรหรือข้อความ เป็นข้อมูลทั่วๆ ไปที่เรามักจัดเก็บในฐานข้อมูล
CLOB	ชุดของตัวอักษรขนาดใหญ่	ตัวอักษรหรือข้อความ ซึ่งตัวอักษรประเภทนี้สามารถแทนรูปแบบของตัวอักษรได้มากกว่ารหัส ASCII รวมถึงภาษาของตัวอักษร และจัดเก็บข้อความขนาดยาวมาก—Character Large Object
VARCHAR	ชุดของตัวอักษร ASCII มีความยาวแปรผัน	ตัวอักษรหรือข้อความ โดยความยาวของข้อความแปรผันได้ตามข้อมูลจริง แต่จะจัดเก็บค่าได้ยาวที่สุดตามขนาดของคอลัมน์ที่กำหนด
INTEGER	จำนวนเต็มขนาด 32 บิต	จำนวนเต็มขนาด 32 บิตสำหรับแต่ละคอลัมน์
SMALLINT	จำนวนเต็มขนาด 16 บิต	จำนวนเต็มขนาด 16 บิตสำหรับแต่ละคอลัมน์
DECIMAL/ NUMERIC	ตัวเลขทศนิยม	ตัวเลขจำนวนเต็มและทศนิยม
FLOAT	จำนวนจริงขนาดสั้นที่สุด 24 บิต และยาวที่สุด 53 บิต	FLOAT คือจำนวนจริงใดๆ
REAL	จำนวนจริงขนาด 24 บิต	REAL เป็นคำที่ใช้แทน FLOAT(24) ซึ่งเป็นจำนวนจริงขนาดสั้นที่สุด
DOUBLE	จำนวนจริงขนาดยาวที่สุด 53 บิต	DOUBLE เป็นคำที่ใช้แทน FLOAT(53) ซึ่งเป็นจำนวนจริงขนาดยาวที่สุด
DATE	วันที่	ใช้จัดเก็บวันที่ ปีใน SQL จัดเก็บในรูปแบบ YYYY จึงไม่มีปัญหากับปี 2000 และ SQL ต้องสามารถคำนวณปีอธิกสุรทินหรือปีที่มีเดือนกุมภาพันธ์ 29 วันได้อย่างถูกต้อง
TIME	เวลา	เวลาใน SQL บรรจุข้อมูลวินาทีกำกับไว้เสมอรวมทั้งอาจมีส่วนของวินาทีและโซนของเวลา
TIMESTAMP	วันที่ และ เวลา	การนำวันที่และเวลามาเขียนต่อกัน
INTERVAL	ระยะห่างระหว่างวันที่ เวลา หรือ TIMESTAMP 2 ค่า	ระยะห่างระหว่างวันที่ เวลา หรือ TIMESTAMP 2 ค่า แต่จะต้องป้อนค่าตาม syntax ที่กำหนด

ซึ่งข้อมูลที่ใช้กันโดยทั่วไปสามารถแบ่งออกเป็น 5 ประเภทใหญ่ๆ ได้แก่

1) ตัวอักษร และ ข้อความ
2) ตัวเลขที่สามารถนำมาใช้ได้ในการคำนวณ เช่น จำนวนนับ เงินตรา หรือจำนวนจริง เป็นต้น
3) เวลา และ วันที่
4) ตรรกะ เช่น จริง/ไม่จริง ถูก/ผิด เปิด/ปิด เป็นต้น
5) วัตถุ (object) อื่นๆ เช่นไฟล์เอกสารต่างๆ รูปภาพ ไฟล์เสียง เป็นต้น

จากตารางที่ 7-1 VARCHAR ไม่ได้รองรับด้วย SQL:1999 แต่เป็นชนิดของข้อมูลที่สำคัญและใช้สำหรับสายอักขระส่วนใหญ่ในปัจจุบัน จึงนำมาแสดงไว้ สิ่งสำคัญที่ VARCHAR ต่างจาก CHAR เกี่ยวข้องกับการแสดงผล การแสงดผลของ CHAR นั้น SQL จะไม่ทราบว่าข้อความที่บรรจุอยู่ในคอลัมน์โดยแท้จริงนั้นยาวเท่าใด ผลลัพธ์ที่ได้จาก SQL ในการแสดงผลข้อมูล CHAR นั้นจะมีความยาวคงที่ กล่าวคือแม้ข้อความจะสั้นกว่าขนาดของคอลัมน์ที่ได้ประกาศไว้ หลังจากข้อความจะมีช่องว่างเป็นขนาดเท่ากับจำนวนตัวอักษรที่เหลืออยู่ของคอลัมน์ ซึ่งต่างจาก VARCHAR ที่จะแสดงข้อความพอดีกับขนาดของข้อมูลจริง การจัดการข้อมูล CHAR ใช้ทรัพยากรน้อยกว่า VARCHAR ดังนั้นในกรณีที่ข้อมูลมีความยาวที่แน่นอนจึงควรใช้ CHAR

ในทางปฏิบัติแล้วระบบจัดการฐานข้อมูลเชิงสัมพันธ์ของผู้ผลิตแต่ละรายจะรองรับชนิดของข้อมูลต่างกันออกไปอีก ตามแต่จะรายจะเพิ่มความสามารถและประสิทธิภาพของระบบจัดการฐานข้อมูลของตนเอง เช่น อาจมีการรองรับชนิดข้อมูลประเภท CURRENCY สำหรับรองรับข้อมูลทางด้านการเงินซึ่งสามารถจัดเก็บรูปแบบการแสดงผล ตลอดจนสกุลเงินได้ หรือการรองรับตัวเลขที่มีขนาดใหญ่มากด้วยชนิดของข้อมูลพิเศษอื่นๆ เป็นต้น

7.4 การสร้างและจัดการสคีมา

การสร้างและจัดการสคีมาเป็นการจัดการเฉพาะโครงสร้างของฐานข้อมูล เช่นการสร้างและการลบฐานข้อมูลด้วยคำสั่ง CREATE DATABASE และ DROP DATABASE ในหัวข้อ 7.4.1 การสร้างและการลบตารางข้อมูลด้วยคำสั่ง CREATE TABLE และ DROP TABLE ในหัวข้อ 7.4.2 และ 7.4.4 ตามลำดับ การสร้างการเปลี่ยนแปลงโครงสร้างตารางด้วยคำสั่ง ALTER TABLE ในหัวข้อ 7.4.3 เป็นคำสั่ง SQL ในกลุ่มนิยามข้อมูล—DDL ในกรณีที่ไม่ได้ระบุเป็นอย่างอื่น คำสั่ง SQL ที่แสดงอ้างอิงรูปแบบของคำสั่ง SQL-92 ซึ่งเป็นมาตรฐานของ DDL และ DML ที่ใช้กันในปัจจุบัน ตัวอย่างที่จะกล่าวถึงในหัวข้อถัดไป ใช้โครงสร้างของฐานข้อมูล University (Ramakrishnan and Gehrke, 2003) ซึ่งอ้างอิงถึงบางตารางของสคีมาต่อไปนี้

Student (*sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)

Faculty (*fid:* string, *fname:* string, *sal:* real)

Course (*cid:* string, *cname:* string, *credits:* integer)

Room (*rno:* integer, *address:* string, *capacity:* integer)

Enrolled (*sid:* string, *cid:* string, *grade:* float)

Teaches (*fid:* string, *cid:* string)

Meets_In (*cid:* string, *rno*: integer, *time:* string)

7.4.1 การสร้างฐานข้อมูล

คำสั่งในการสร้างตารางอาจจะต่างกันออกไปสำหรับระบบจัดการฐานข้อมูลของผู้ผลิตแต่ละราย เนื่องจากการจัดการไฟล์ทางกายภาพไม่เหมือนกัน ในที่นี้ใช้รูปแบบอย่างง่ายของระบบจัดการฐานข้อมูล Microsoft SQL Server ซึ่งใช้ในปฏิบัติการ

```
CREATE DATABASE <database_name>
[ON PRIMARY
(
        NAME= '<database_name >',
        FILENAME='<data_file_name>'
        [, SIZE=<data_file_size>]
        [, MAXSIZE=<data_file_max_size>]
        [, FILEGROWTH=<data_file_growth_step>]
)
LOG ON
(
        NAME= '<log_name >',
        FILENAME='<log_file_name>'
        [, SIZE=<log_file_size>]
        [, MAXSIZE=<log_file_max_size>]
        [, FILEGROWTH=<log_file_growth_step>]
)];
```

CREATE DATABASE	คำสั่งสำหรับสร้างฐานข้อมูล
database_name	ชื่อของฐานข้อมูลซึ่งจะใช้อ้างอิงต่อไปใน **DBMS**
ON PRIMARY	ระบุรายละเอียดการสร้างไฟล์หลักด้วยตนเอง ซึ่งคำสั่งที่อธิบายต่อไปนี้ไม่จำเป็นต้องกำหนด
NAME	ชื่อของวัตถุในฐานข้อมูล ในที่นี้คือชื่อไฟล์ที่จัดเก็บข้อมูลและไฟล์ log
FILENAME	ชื่อของไฟล์ที่จัดเก็บวัตถุฐานข้อมูลนั้นๆ โดยเป็นชื่อทางกายภาพของไฟล์ที่จัดเก็บในระบบปฏิบัติการ มีนามสกุลเป็น mdf (ไฟล์ข้อมูล) และ ldf (ไฟล์ log)
SIZE	ขนาดเริ่มต้นของไฟล์วัตถุในฐานข้อมูล มีหน่วยเป็น MB
MAXSIZE	ขนาดที่ใหญ่ที่สุดที่อนุญาตให้ไฟล์ของวัตถุในฐานข้อมูลสามารถขยายได้ มีหน่วยเป็น MB
FILEGROWTH	ขนาดการขยายไฟล์วัตถุในฐานข้อมูลแต่ละครั้ง มีหน่วยเป็น MB
LOG ON	เป็นการระบุการจัดเก็บ log หรือการบันทึกการทำงานของฐานข้อมูล log นี้มีประโยชน์สำหรับการตรวจสอบการทำงานตลอดจนการจัดการธุรกรรมและการกู้คืนข้อมูลซึ่งจะได้อธิบายในบทที่ 9

ตัวอย่างที่ 7.1 การสร้างฐานข้อมูล (CREATE TABLE)

สร้างฐานข้อมูล university โดยไฟล์ที่ใช้จัดเก็บข้อมูลชื่อ university.mdf และไฟล์ Log ชื่อ university.ldf

```
CREATE DATABASE university
ON PRIMARY
(
        NAME='university_DAT',
        FILENAME='C:\SQL\university.mdf',
        SIZE=4,
        MAXSIZE=20,
        FILEGROWTH=1
)
LOG ON
(
        NAME= 'university_LOG',
        FILENAME='C:\SQL\university.ldf'
        SIZE=2,
        MAXSIZE=5,
        FILEGROWTH=1
);
```

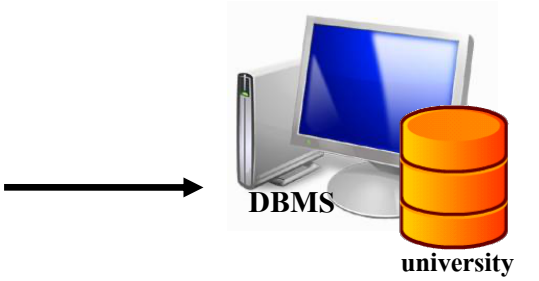
สามารถแสดงแผนภาพประกอบคำอธิบายการประมวลผลคำสั่ง SQL ในการสร้างฐานข้อมูลดังนี้

ก่อนการสร้างฐานข้อมูล

ก่อนการสร้างฐานข้อมูล เครื่องคอมพิวเตอร์ที่ติดตั้งระบบจัดการฐานข้อมูลยังไม่มีฐานข้อมูล university

สร้างฐานข้อมูล

```
CREATE DATABASE university
ON PRIMARY
(
        NAME='university_DAT',
        FILENAME='C:\SQL\university.mdf',
        SIZE=4,
        …
);
```



ชื่อไฟล์ข้อมูล: university.mdf

ชื่อไฟล์ Log: university.ldf

จากตัวอย่างเป็นการสร้างฐานข้อมูลชื่อ university โดยไฟล์ที่จะจัดเก็บข้อมูลต่างๆ ของผู้ใช้จะถูกจัดเก็บไว้ในไฟล์ university.mdf ซึ่งเป็นรูปแบบของ Microsoft SQL Server เท่านั้น รวมถึงไฟล์ที่ใช้จัดเก็บบันทึกรายการกิจกรรมของฐานข้อมูลหรือ Log ในชื่อ university.ldf ทั้ง 2 ไฟล์ถูกวางไว้ที่ "C:\SQL" ของเครื่องคอมพิวเตอร์ที่ระบบจัดการฐานข้อมูลควบคุมอยู่ ระบบจัดการฐานข้อมูลจะทำการประมวลคำสั่ง SQL ที่ใช้ในการสร้างฐานข้อมูลจากนั้นสร้างวัตถุที่เกี่ยวข้อง การสร้างฐานข้อมูลเป็นการกำหนดโครงสร้างของฐานข้อมูลเท่านั้น จึงเป็นการนิยามโครงสร้างของข้อมูล ยังไม่ได้มีการสร้างตารางหรือจัดเก็บข้อมูลที่เราใช้ใดๆ ลงในฐานข้อมูล ในภาพตัวอย่างใช้วัตถุทรงกระบอกแทนฐานข้อมูล คำสั่งสร้างฐานข้อมูลแตกต่างกัน

ออกไปตามระบบจัดการฐานข้อมูลและเป็นคำสั่งที่ใช้งานไม่บ่อยนักเนื่องจากระบบสารสนเทศแต่ละระบบมักใช้งานฐานข้อมูลเพียงฐานข้อมูลเดียวหรือมีจำนวนไม่มาก แต่เรียกใช้งานข้อมูลจากหลายๆ ตาราง

การลบฐานข้อมูลสามารถทำได้โดยการเรียกใช้คำสั่ง DROP DATABASE ดังนี้

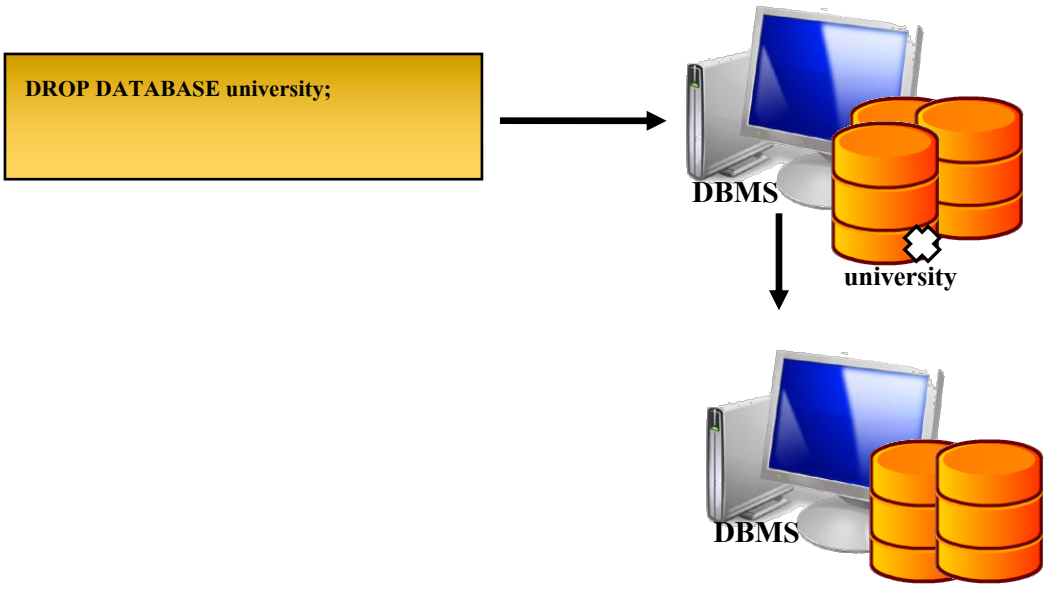
```
DROP DATABASE <database_name>;
```

ตัวอย่างที่ 7.2 การลบฐานข้อมูล (DROP DATABASE)

ลบฐานข้อมูล university

DROP DATABASE university;

สามารถแสดงแผนภาพประกอบคำอธิบายการประมวลผลคำสั่ง SQL ในการลบฐานข้อมูลดังนี้



จากตัวอย่าง ระบบคอมพิวเตอร์อาจมีฐานข้อมูลบรรจุอยู่มากกว่า 1 ฐานข้อมูล ในภาพจำลองนี้จำลองฐานข้อมูลเป็น 3 ฐานข้อมูล ซึ่ง 1 ในนั้นได้แก่ฐานข้อมูล University เมื่อระบบจัดการฐานข้อมูลประมวลผลคำสั่งลบฐานข้อมูลแล้ว ฐานข้อมูล University จะถูกลบทิ้ง คงเหลืออีกเพียง 2 ฐานข้อมูล

7.4.2 การสร้างตาราง

คำสั่งสร้างตารางเป็นคำสั่งสำคัญสำหรับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ เนื่องจากเอนทิตีและความสัมพันธ์ต้องถูกแทนด้วยตาราง รูปแบบอย่างง่ายของคำสั่งที่ใช้ในการสร้างตารางแสดงได้ดังนี้

```
CREATE TABLE <table_name>
(
        <column_name data_type>
        [,<column_name data_type>]…
);
```

โดย

CREATE TABLE	คือคำสั่งที่ใช้ในการสร้างตารางในฐานข้อมูล
table_name	ชื่อของตารางที่ต้องการสร้าง
column_name	ชื่อของคอลัมน์ในตาราง
data_type	ชนิดของข้อมูลที่สัมพันธ์กับคอลัมน์ที่สร้างขึ้นในตาราง

ตารางหนึ่งตารางสามารถมีหลายคอลัมน์ได้โดยการแสดงรายการคอลัมน์และชนิดของคอลัมน์ ขั้นคอลัมน์แต่ละคอลัมน์ด้วยเครื่องหมายจุลภาค “,” สามารถเขียนรูปแบบคำสั่งสร้างตารางที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

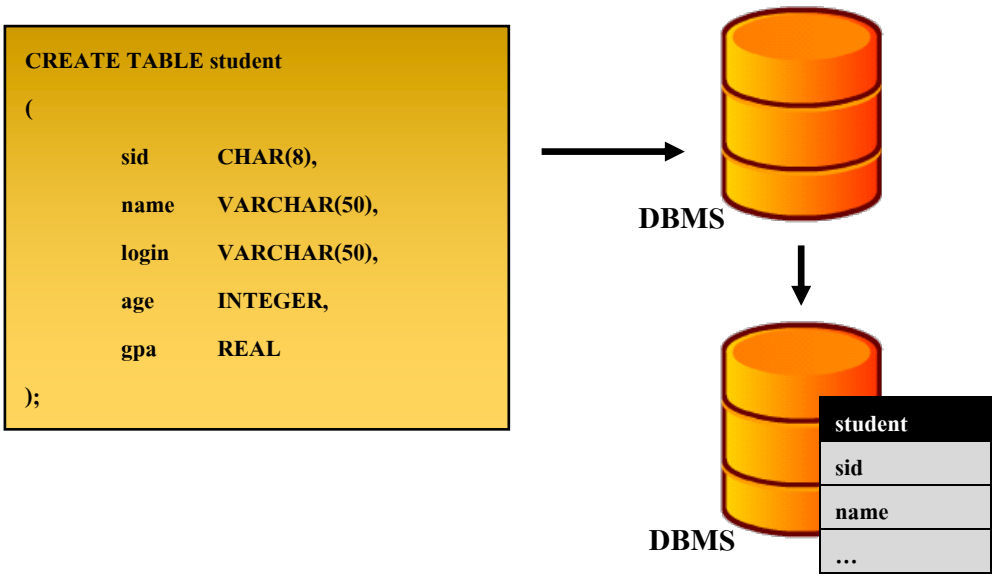
```
สร้าง ตาราง <ชื่อตาราง>
(
        <ชื่อคอลัมน์ ประเภทข้อมูล>
        [,<ชื่อคอลัมน์ ประเภทข้อมูล>]…
);
```

ตัวอย่างที่ 7.3 การสร้างตาราง (CREATE TABLE)

สร้างตารางนักศึกษา ซึ่งมีสคีมาดังนี้ Student (*sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)

```
CREATE TABLE student
(
        sid     CHAR(8),
        name    VARCHAR(50),
        login   VARCHAR(50),
        age     INTEGER,
        gpa     REAL
);
```

สามารถแสดงแผนภาพประกอบคำอธิบายการประมวลผลคำสั่ง SQL ในการสร้างตารางข้อมูลดังนี้



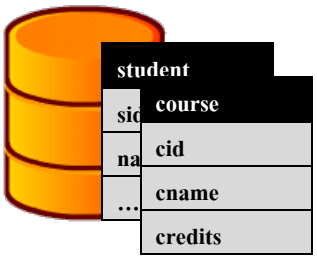
จากตัวอย่างเป็นสร้างตาราง student ประกอบไปด้วยคอลัมน์ 5 คอลัมน์ได้แก่ sid, name, login, age และ gpa ซึ่งคอลัมน์ 3 คอลัมน์แรกนั้นมีชนิดของข้อมูลเป็น string ซึ่งในระบบจัดการฐานข้อมูลโดยทั่วไปใช้ฐานข้อมูล CHAR และ VARCHAR ในการจัดการกับข้อมูลสายอักขระ—string นี้ สำหรับชนิดของข้อมูลที่สามารถกำหนดขนาดของคอลัมน์ได้นั้น สามารถกำหนดไว้ในเครื่องหมายวงเล็บ ( ) หลังชนิดของข้อมูล จากตัวอย่าง sid เป็นชื่อของคอลัมน์แทนรหัสนักศึกษามีชนิดของข้อมูลเป็น CHAR ความยาว 8 ตัวอักษร ซึ่งสามารถจัดเก็บข้อมูลได้ความยาวสูงสุด 8 ตัวอักษร และความยาวมีค่าคงที่เช่น ‘B5075666’ ในกรณีที่รหัสนักศึกษามีค่าสั้นกว่า 8 ตัวอักษร ระบบจัดการฐานข้อมูลจะแทนค่าตัวอักษรที่ว่างอยู่ด้วยช่องว่าง เช่น ข้อมูล ‘B507’ ในคอลัมน์ CHAR ระบบจัดการฐานข้อมูลจะจัดการและแสดงในรูป “B507 ” ซึ่งช่องว่างนั้นมีขนาด 4 ตัวอักษรพอดี สำหรับ name และ login เป็นชื่อของคอลัมน์แทนชื่อนักศึกษาและชื่อสำหรับการเข้าใช้ระบบตามลำดับ login ทั้ง 2 คอลัมน์นี้จัดเก็บข้อมูลชนิดตัวอักษรความยาวแปรผัน VARCHAR ความยาวสูงสุด 50 ตัวอักษร เช่น ‘สมชาย’ ข้อมูลชนิด VARCHAR นี้ระบบจัดการฐานข้อมูลจะแสดงผลลัพธ์ที่มีความยาวเท่ากับความยาวของข้อมูลจริงที่บรรจุอยู่ในคอลัมน์ไม่ใช่ตามความยาวของขนาดคอลัมน์ดังที่ CHAR แสดง ความยาวสูงสุดของสายอักขระผู้สร้างตารางต้องระบุเองโดยกำหนดจากการวิเคราะห์ให้สอดคล้องกับการใช้งานจริง age และ gpa เป็นข้อมูลชนิดจำนวนเต็มและจำนวนจริง แทนอายุและเกรดเฉลี่ยของนักศึกษาตามลำดับ

จากแผนภาพ ระบบจัดการฐานข้อมูล—DBMS เป็นผู้ประมวลผลคำสั่ง SQL ซึ่งในแผนภาพได้ละ DBMS ไว้ แสดงแต่เพียงฐานข้อมูล university แทนด้วยทรงกระบอก ควรระลึกไว้เสมอว่าฐานข้อมูลไม่สามารถเข้าถึงได้โดยตรง แต่ต้องใช้งานด้วยคำสั่ง SQL ผ่านทาง DBMS เสมอเพื่อคงประโยชน์และความปลอดภัยสูงสุดของระบบจัดการฐานข้อมูล เมื่อหน่วยประมวลคำสั่ง SQL และหน่วยจัดการข้อมูลอื่นๆ ใน DBMS ประมวลผลสำเร็จ จะเกิดโครงสร้างของตารางข้อมูลขึ้นในฐานข้อมูลที่ DBMS จัดการอยู่ในปัจจุบัน ซึ่งในที่นี้คือเกิดตาราง student ในฐานข้อมูล university นั่นเอง โดยตาราง student นี้ยังไม่มีข้อมูลใดๆ มีเพียงชื่อของตาราง คอลัมน์ ชนิดของคอลัมน์ซึ่งเปรียบเสมือนมีเพียงโครงสร้างข้อมูลรอการจัดเก็บข้อมูลต่อไป ตัวอย่างต่อไปนี้แสดงการสร้างตารางอื่นๆ เพิ่มเติม

ตัวอย่างที่ 7.4 การสร้างตาราง (CREATE TABLE)

สร้างตารางรายวิชาจากสคีมา Course (*cid:* string, *cname:* string, *credits:* integer)

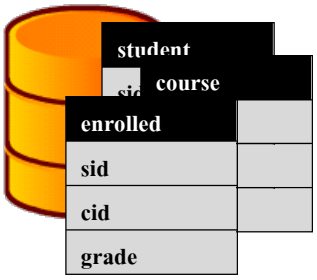
```
CREATE TABLE course
(
        cid       CHAR(6),
        cname     VARCHAR(100),
        credits   INTEGER
);
```



ตัวอย่าง 7.5 การสร้างตาราง

สร้างตารางการลงทะเบียนจากสคีมา Enrolled (*sid:* string, *cid:* string, *grade:* float)

```
CREATE TABLE enrolled
(
        sid       CHAR(8),
        cid       CHAR(6),
        grade     FLOAT
);
```



ทั้ง 2 ตัวอย่างหลังเป็นสร้างตารางเพิ่มจากเดิมที่มีอยู่เพียงตาราง student

7.4.3 การแก้ไขตาราง

เมื่อมีการสร้างตารางจากข้อกำหนดของระบบหรือโมเดลเชิงตรรกะของระบบแล้วเป็นเรื่องปกติที่ข้อกำหนดของระบบอาจมีการเปลี่ยนแปลง เช่น ระบบลงทะเบียนต้องสามารถระบุสังกัดของนักศึกษาได้ว่าสังกัดสาขาใด สคีมาของตารางนักศึกษาจึงต้องจัดเก็บรหัสของสาขาวิชาด้วย เป็นต้น หรืออาจมีการปรับเปลี่ยนโครงสร้างของคอลัมน์ด้วยเหตุผลอื่นๆ การแก้ไขโครงสร้างของตารางจึงเป็นเรื่องปกติ คำสั่งที่ใช้ในการแก้ไขโครงสร้างตารางมีรูปแบบอย่างง่ายดังนี้

```
ALTER TABLE <table_name>
<alter_commands> <column_name> [data_type];
```

โดย

ALTER TABLE	คือคำสั่งที่ใช้ในการแก้ไขตาราง
table_name	ชื่อของตารางที่ต้องการแก้ไข จะต้องเป็นตารางที่มีอยู่แล้วในฐานข้อมูล
alter_commands	เป็นคำสั่งในการแก้ไขตาราง ได้แก่ ADD, ALTER และ DROP สำหรับเพิ่ม แก้ไข และลบคอลัมน์หรือวัตถุใดๆ ในตารางข้อมูล
column_name	ชื่อของคอลัมน์ในตารางที่ต้องการแก้ไข
data_type	ชนิดของข้อมูลใหม่ที่สัมพันธ์กับคอลัมน์ที่แก้ไข

สามารถเขียนรูปแบบคำสั่งแก้ไขตารางที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

แก้ไข ตาราง <ชื่อตาราง>

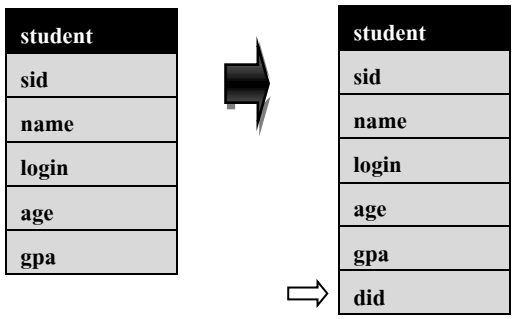
<คำสั่งแก้ไขข้อมูล> <ชื่อคอลัมน์> [<ประเภทข้อมูล>];

ตัวอย่างที่ 7.6 การเพิ่มคอลัมน์ (ADD COLUMN)

แก้ไขตารางนักศึกษา โดยการ**เพิ่ม**คอลัมน์รหัสสาขา did เป็นข้อมูลชนิดสายอักขระความยาว 3 ตัวอักษร

ALTER TABLE student

ADD did CHAR (3);



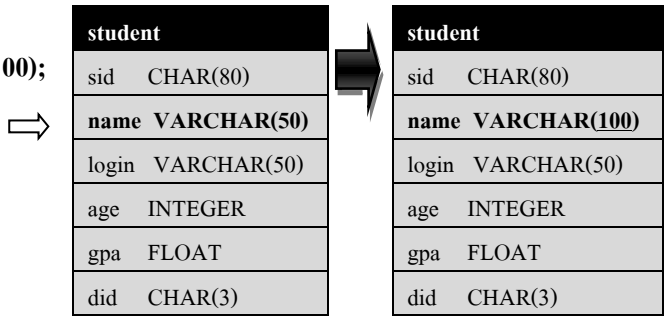
จากตัวอย่าง ส่วนของคำสั่ง **ALTER TALBE student** เป็นการระบุให้ทำการแก้ไขโครงสร้างของตารางนักศึกษา และ **ADD** ใน **(ADD did CHAR(3))** ทำการเพิ่มคอลัมน์ที่ชื่อว่า did เป็นชนิด CHAR ความยาว 3 ตัวอักษร คำสั่ง ADD ยังใช้ในการเพิ่มวัตถุอื่นๆ นอกจากคอลัมน์ โดยจะได้กล่าวถึงในหัวข้อต่อๆ ไป

ตัวอย่างที่ 7.7 การแก้ไขคอลัมน์ (ALTER COLUMN)

แก้ไขตารางนักศึกษา โดยการ**แก้ไข**คอลัมน์ชื่อนักศึกษา name ให้เป็นข้อมูลชนิด VARCHAR ที่รองรับสายอักขระได้ยาวถึง 100 ตัวอักษร จากเดิม 50 ตัวอักษร

ALTER TABLE student

ALTER COLUMN name VARCHAR(100);



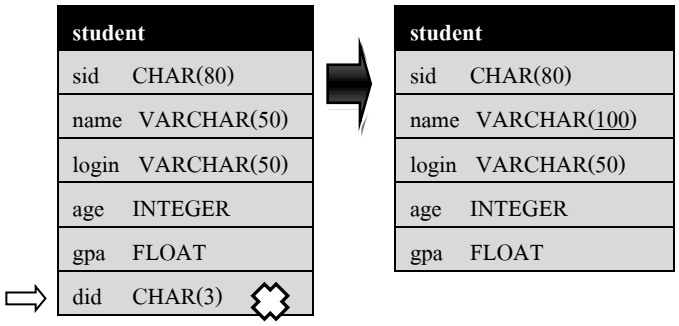
จากตัวอย่าง ส่วนของคำสั่ง **ALTER TALBE student** เป็นการระบุให้ทำการแก้ไขโครงสร้างของตารางนักศึกษา และ **ALTER COLUMN** ใน **(ALTER COLUMN name VARCHAR(100))** ทำการแก้ไขคอลัมน์ที่ชื่อว่า name ให้เป็นชนิด VARCHAR ความยาว 100 ตัวอักษร เช่นเดียวกันกับคำสั่ง ADD คำสั่ง ALTER ยังสมารถใช้ในการแก้ไขวัตถุอื่นๆ นอกจากคอลัมน์ โดยจะได้กล่าวถึงในหัวข้อต่อๆ ไป

ตัวอย่างที่ 7.8 การลบคอลัมน์ (DROP COLUMN)

แก้ไขตารางนักศึกษา โดยการ**ลบ**คอลัมน์รหัสสาขา did ทิ้งเสีย

ALTER TABLE student

DROP did;



จากตัวอย่าง ส่วนของคำสั่ง **ALTER TALBE student** เป็นการระบุให้ทำการแก้ไขโครงสร้างของตารางนักศึกษา และ **DROP** ใน (DROP did) ทำการลบคอลัมน์ที่ชื่อว่า did ทิ้งไป

7.4.4 การลบตาราง

คำสั่งสร้างตารางเป็นคำสั่งสำคัญสำหรับระบบจัดการฐานข้อมูลเชิงสัมพันธ์ เนื่องจากเอนทิตีและความสัมพันธ์ต้องถูกแทนด้วยตาราง รูปแบบอย่างง่ายของคำสั่งที่ใช้ในการสร้างตารางแสดงได้ดังนี้

```
DROP <column_name>;
```

โดย

DROP คือคำสั่งที่ใช้ในการลบตารางออกจากฐานข้อมูล

column_name ชื่อของคอลัมน์ที่ต้องการลบในตาราง

สามารถเขียนรูปแบบคำสั่งลบตารางที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

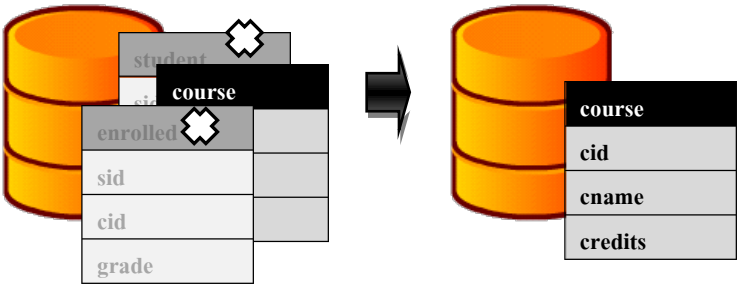
ลบ <ชื่อตาราง>;

ตัวอย่างที่ 7.9 การลบตาราง (DROP TABLE)

ลบตาราง student และ enrolled

DROP student;

DROP enrolled;



จากตัวอย่างเป็นการลบตาราง student และ enrolled ทีละตาราง คงเหลือเฉพาะตาราง course ดังภาพ การ DROP ตารางจะทำให้ข้อมูลในตารางหายไปด้วยในกรณีที่มีข้อมูลบรรจุอยู่ในตาราง คำสั่ง DROP สามารถใช้ลบวัตถุอื่นๆ ของฐานข้อมูลได้ เช่น เงื่อนไขบังคับบูรณภาพ—Integrity Constraints ต่างๆ

7.5 บูรณภาพของข้อมูล Data Integrity

บูรณภาพของข้อมูลคือความถูกต้องของข้อมูลทั้งฐานข้อมูลดังได้อธิบายในหัวข้อ 3.4 ในบทที่ 3 และ 4.3 ในบทที่ 4 และกล่าวถึงอีกหลายครั้งเนื่องจากมีความสำคัญและเป็นประโยชน์ที่สำคัญที่ได้รับจากระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ในที่นี้จะนำตัวอย่างในหัวข้อ 4.3 จากบทที่ 4 มาแสดงอีกครั้งโดยย่อ ซึ่งสามารถอ้างอิงได้จากหัวข้อที่เกี่ยวข้องกับบูรณภาพของข้อมูลในบทที่ 3 และ 4 โดยใช้สคีมาและกรณีตัวอย่างข้อมูลของตาราง student และ enrolled ต่อไปนี้ (ตาราง student จากหัวข้อ 4.3 ในบทที่ 4 มี 5 คอลัมน์)

Student (*cid: string, sid:* string, *name:* string, *login:* string, *age:* integer, *gpa:* real)

Enrolled (*sid:* string, *cid:* string, *grade:* float)

1) รหัสนักศึกษาในตาราง enrolled ต้องปรากฏอยู่ในตาราง student

Enrolled

cid	grade	sid
204203	A	B5075650
204204	B	B5075831
204205	D	B5075688
204329	F	B5075666

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

2) รหัสนักศึกษาจำเป็นต้องมีค่าอยู่ในคอลัมน์ มีค่าไม่ซ้ำกัน และใช้เป็นคีย์หลัก

4) login ของนักศึกษาแต่ละคนห้ามมีค่าซ้ำกัน

3) เกรดเฉลี่ยต้องมีค่าระหว่าง 0 ถึง 4.00

ภาพที่ 7-1 เงื่อนไขบังคับบูรณภาพแบบต่างๆ

จากตัวอย่าง ข้อมูลที่บรรจุในฐานข้อมูลจะมีความถูกต้องหรือมีบูรณภาพได้จะต้องเป็นไปตามข้อกำหนดของฐานข้อมูลเพื่อคงความถูกต้องของฐานข้อมูล เช่น หมายเลข 1) รหัสนักศึกษาในตาราง enrolled ต้องปรากฏอยู่ในตารางstudent การกำหนดดังนี้เพื่อป้องกันความผิดพลาดของการเพิ่มระเบียนในตาราง enrolled หากมีการเพิ่มข้อมูลโดยบรรจุระเบียนการลงทะเบียนเป็นรหัสนักศึกษาที่ไม่ปรากฏอยู่ในตารางนักศึกษา เป็นการเพิ่มข้อมูลที่ไม่ถูกต้อง ทำให้ข้อมูลขาดบูรณภาพ สำหรับหมายเลข 2), 3) และ 4) ต่างก็เป็นข้อกำหนดอื่นๆ เพื่อให้ข้อมูลมีความถูกต้องตรงกับการใช้งาน ข้อกำหนดที่ได้ยกตัวอย่างนี้เป็นเพียงส่วนหนึ่งของข้อกำหนดที่จำเป็นต้องระบุไว้ในตาราง student และ enrolled ดังแสดง

ข้อกำหนดที่ได้ยกตัวอย่างแต่ละข้ออาจแยกออกเป็นข้อกำหนดย่อยๆ ได้ ซึ่งเราจะต้องกำหนดเงื่อนไขในการแก้ไขข้อมูลเพื่อให้ข้อมูลมีความถูกต้องตรงตามข้อกำหนด เงื่อนไขต่างๆ เหล่านี้เรียกว่าเงื่อนไขบังคับบูรณภาพ (integrity constraints) เงื่อนไขบังคับบูรณภาพเหล่านี้จัดอยู่ในประเภทต่างๆ เช่น เงื่อนไขบังคับคีย์ เงื่อนไขบังคับคีย์หลัก เงื่อนไขบังคับการมีส่วนร่วม เงื่อนไขบังคับคีย์ภายนอก เงื่อนไขบังคับทั่วไป เป็นต้น เราสามารถกำหนดเงื่อนไขบังคับเหล่านี้ให้กับโครงสร้างข้อมูลในฐานข้อมูลด้วย SQL ได้ด้วยเงื่อนไขบังคับต่อไปนี้

PRIMARY KEY	เงื่อนไขบังคับคีย์หลัก เป็นการบังคับว่าค่าในคอลัมน์ที่กำหนดนั้นเป็นคีย์หลัก ซึ่งจะต้องมีค่าไม่ซ้ำกัน และไม่เป็นค่าว่างด้วย
NOT NULL	เงื่อนไขบังคับค่าว่าง เป็นการบังคับว่าคอลัมน์นั้นๆ ห้ามเป็นค่าว่าง
DEFAULT	เงื่อนไขบังคับค่าโดยปริยาย เป็นการกำหนดค่าโดยปริยายหากไม่มีการระบุค่าของคอลัมน์นั้นๆ
UNIQUE KEY	เงื่อนไขบังคับคีย์ เป็นการบังคับว่าค่าในคอลัมน์ที่กำหนดต้องมีค่าไม่ซ้ำกัน
FORIGN KEY	เงื่อนไขบังคับเชิงอ้างอิง เป็นการบังคับว่าค่าในคอลัมน์ของตารางหนึ่งจะต้องปรากฏอยู่ในค่าในคอลัมน์ของอีกตารางหนึ่ง
CHECK	เงื่อนไขบังคับตรวจสอบ เป็นการบังคับว่าค่าในคอลัมน์จะต้องเท่ากับค่าหรืออยู่ในช่วงค่าที่กำหนด

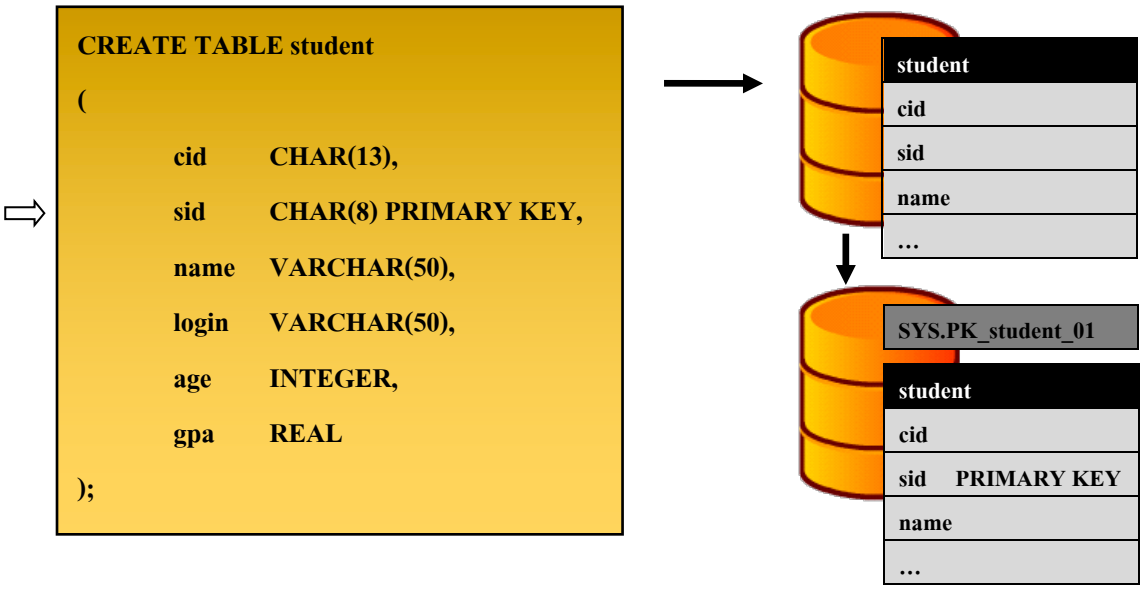
เงื่อนไขบังคับบูรณภาพที่ได้จากการออกแบบฐานข้อมูลในระดับแนวคิดและระดับตรรกะนั้นจะต้องใช้เงื่อนไขบังคับของ SQL ที่เหมาะสมในการประยุกต์กับฐานข้อมูลจริง ในบางกรณีอาจจะต้องใช้หลายเงื่อนไขบังคับประกอบกัน การกำหนดเงื่อนไขบังคับแต่ละเงื่อนไขมีรายละเอียดปลีกย่อยพอสมควร ให้พิจารณารูปแบบการใช้งานจากตัวอย่างต่อไปนี้

ตัวอย่างที่ 7.10 การกำหนดเงื่อนไขบังคับคีย์หลัก (PRIMARY KEY CONSTRAINT)

การกำหนดเงื่อนไขบังคับคีย์หลัก โดยกำหนดให้คอลัมน์ sid ในตาราง student เป็นคีย์หลัก

```
CREATE TABLE student
(
        cid     CHAR(13),
        sid     CHAR(8) PRIMARY KEY,
        name    VARCHAR(50),
        login   VARCHAR(50),
        age     INTEGER,
        gpa     REAL
);
```

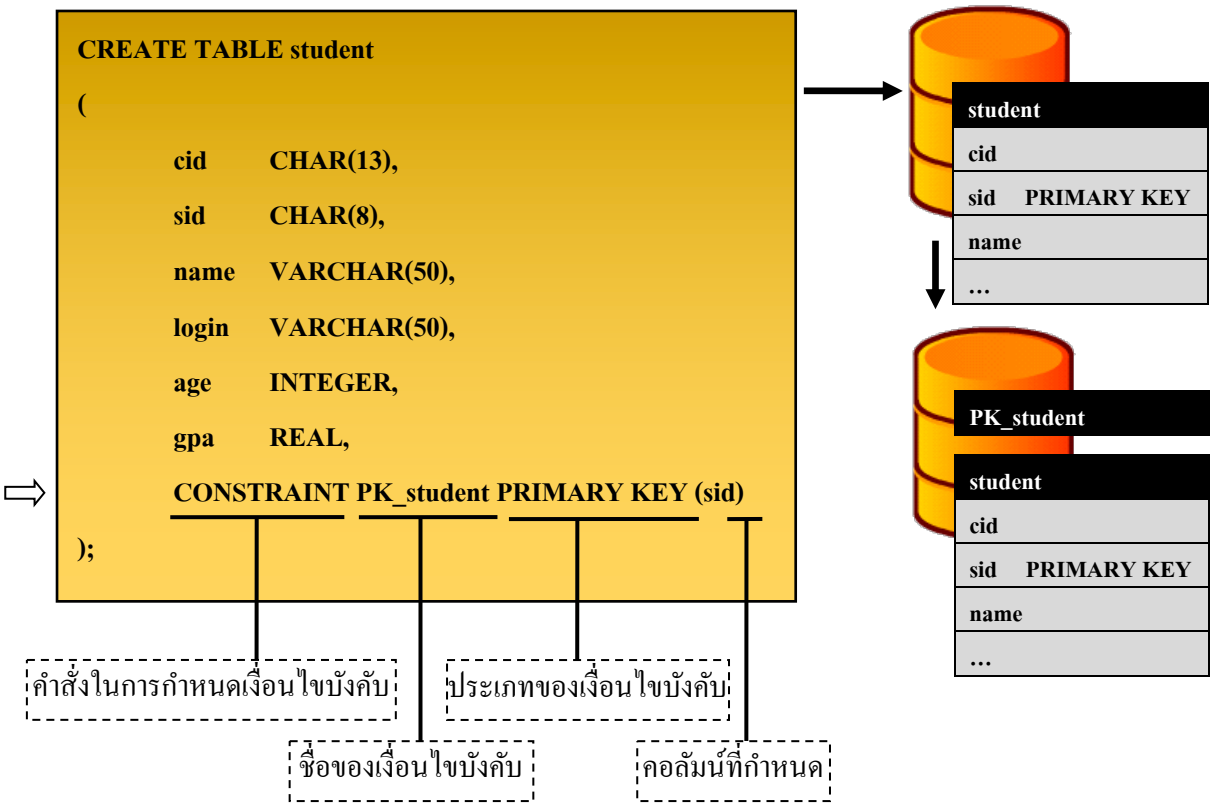
สามารถแสดงแผนภาพประกอบคำอธิบายการประมวลผลคำสั่ง SQL ในการสร้างเงื่อนไขบังคับดังนี้



จากตัวอย่าง การกำหนดเงื่อนไขบังคับดังกล่าวทำให้คอลัมน์ sid ไม่สามารถเป็นค่าว่างและมีค่าซ้ำกันไม่ได้ ซึ่งตรงกับหมายเลข 1) ในกรณีตัวอย่างเงื่อนไขบังคับบูรณภาพ ที่ได้ยกตัวอย่าในภาพที่ 7-1 การกำหนดเงื่อนไขบังคับที่แสดงจำเป็นต้องกำหนดพร้อมๆ กับการสร้างตารางข้อมูลโดยคำสั่ง CREATE TABLE โดยวางประเภทของเงื่อนไขบังคับ PRIMARY KEY ไว้หลังจากการกำหนดชนิดข้อมูลของคอลัมน์ที่ระบุให้เป็นคีย์หลักในที่นี้คือ sid CHAR(8) ระบบจัดการฐานข้อมูลจะกำหนดโครงสร้างในฐานข้อมูลให้คอลัมน์ sid เป็นคีย์หลัก ซึ่งต่อไปนี้การเพิ่มระเบียนข้อมูลลงในตาราง student แต่ละระเบียนนั้น ค่าของคอลัมน์ sid แต่ละระเบียนจะเป็นค่าว่างและมีค่าซ้ำกันไม่ได้

ตัวอย่างที่ 7.11 การกำหนดเงื่อนไขบังคับคีย์หลัก (PRIMARY KEY CONSTRAINT)

เราสามารถกำหนดเงื่อนไขบังคับพร้อมๆ กับการสร้างตารางข้อมูลในอีกรูปแบบหนึ่งดังนี้



จากตัวอย่าง การกำหนดเงื่อนไขบังคับไม่จำเป็นต้องระบุประเภทของเงื่อนไขบังคับไว้ด้านหลังคอลัมน์ที่ต้องการกำหนดเงื่อนไขบังคับก็ได้ มีรูปแบบดังแสดง โดยเพิ่มบรรทัดใหม่หลังจากระบุโครงสร้างคอลัมน์ของตารางข้อมูลเรียบร้อยแล้ว การกำหนดเงื่อนไขบังคับรูปแบบนี้มีข้อดีคือผู้จัดการฐานข้อมูลสามารถจัดการเงื่อนไขบังคับได้สะดวกกว่าในแบบแรกเนื่องจากมีการกำหนดชื่อของเงื่อนไขบังคับนั้นๆ (ระบบจัดการฐานข้อมูลจะกำหนดชื่อของเงื่อนไขบังคับให้ในกรณีที่ไม่ได้กำหนดชื่อด้วยตนเอง) เราสามารถใช้รูปแบบที่คล้ายคลึงกันนี้ในการสร้างเงื่อนไขบังคับประเภทอื่นๆ ได้อีก จึงสามารถพิจารณารูปแบบการใช้งานการสร้างเงื่อนไขบังคับนี้เพื่อประยุกต์ใช้กับการสร้างเงื่อนไขบังคับแบบอื่นๆ โดย

CONSTRAINT คำสั่งในการกำหนดเงื่อนไขบังคับ เป็นคำสั่งที่ต้องคงไว้สำหรับการสร้างเงื่อนไขบังคับทุกๆ ประเภท

PK_student ชื่อของเงื่อนไขบังคับ ในที่นี้คือชื่อของเงื่อนไขบังคับคีย์หลักของตารางนักศึกษา จากแผนภาพด้านบนแสดงให้เห็นว่าโดยแท้จริงแล้วเงื่อนไขบังคับคีย์หลักนี้ถูกสร้างขึ้นเป็นวัตถุหนึ่งในฐานข้อมูล ซึ่งวัตถุนี้จะถูกเรียกใช้งานจากระบบจัดการฐานข้อมูลเพื่อตรวจสอบว่าค่าที่เพิ่มลงในตารางและคอลัมน์ที่เงื่อนไขบังคับนั้นถูกกำหนดขึ้นเป็นไปตามเงื่อนไขหรือไม่

PRIMARY KEY ประเภทของเงื่อนไขบังคับ สามารถแทนด้วย NOT NULL, DEFAULT, UNIQUE KEY, FOREIGN KEY หรือ CHECK ขึ้นอยู่กับประเภทของเงื่อนไขบังคับที่ต้องการสร้าง

sid คอลัมน์ที่กำหนด เป็นคอลัมน์ที่เงื่อนไขบังคับที่สร้างขึ้นนั้น ตรวจสอบค่า สำหรับเงื่อนไขบังคับบางประเภทอาจมีรูปแบบที่ซับซ้อนกว่านี้ ซึ่งนอกการระบุคอลัมน์ที่เงื่อนไขบังคับนั้นตรวจสอบแล้ว ต้องระบุตารางและคอลัมน์อื่นๆ ที่เกี่ยวข้องด้วย เช่น FOREIGN KEY CONSTRAINT หรืออาจต้องระบุเงื่อนไขเพิ่มเติม เช่น CHECK CONSTRAINT เป็นต้น ซึ่งสามารถศึกษาได้จากตัวอย่างการสร้างเงื่อนไขบังคับในตัวอย่างต่อๆ ไป

เนื่องจากการสร้างเงื่อนไขบังคับในลักษณะนี้มีข้อดีที่สำคัญคืออนุญาตให้เราตั้งชื่อเงื่อนไขบังคับเพื่อจัดการในภายหลังได้ ข้อควรคำนึงถึงที่สำคัญประการหนึ่งของการสร้างเงื่อนไขบังคับในลักษณะนี้ ได้แก่การตั้งชื่อของเงื่อนไขบังคับนั้นนิยมตั้งตามมาตรฐานของการตั้งชื่อเงื่อนไขบังคับ เพื่อความสะดวกในการจัดการ โดยเรามักใช้มาตรฐานการตั้งชื่อเงื่อนไขบังคับที่คล้ายคลึงกันดังนี้

PK_TableName	UK_ TableName _ ColumnName
NN_TableName_ColumnName	FK_ReferencedTable_ReferencingTable
DF_ TableName _ ColumnName	CK_TableName

ตัวย่อ 2 ตัวแรกแทนเงื่อนไขบังคับ PRIMARY KEY, NOT NULL, DEFAULT, UNIQUE KEY, FOREIGN KEY และ CHECK ตามลำดับ TableName และ ColumnName แทนชื่อของตารางและคอลัมน์ที่เงื่อนไขบังคับนั้นตรวจสอบ

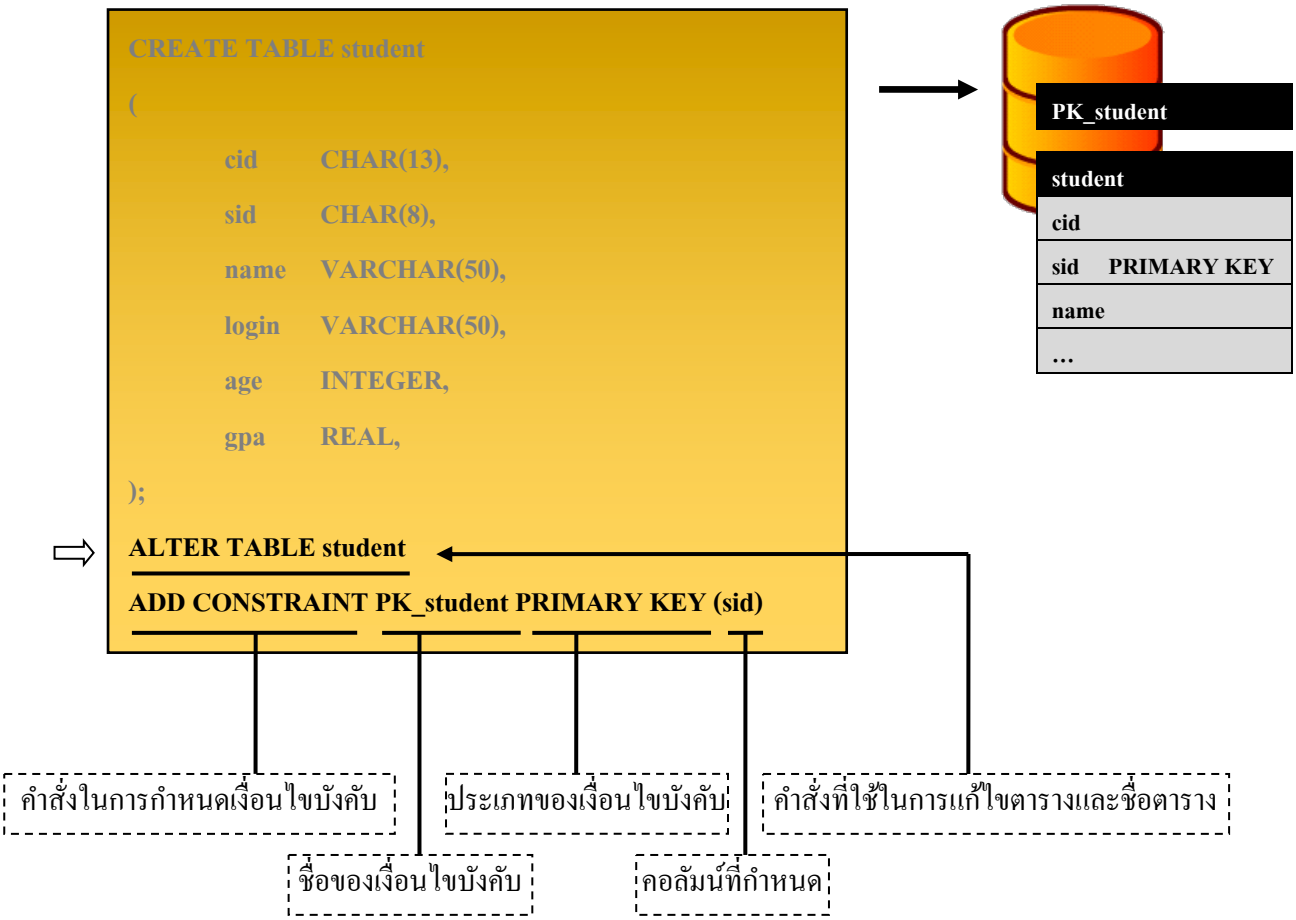
สำหรับเงื่อนไขบังคับ FOREIGN KEY นั้น ReferencedTable คือชื่อของตารางหลักและ ReferencingTable คือชื่อของตารางที่อ้างอิงค่าจากตารางหลัก

ตัวย่อ 2 ตัวแรกมักเขียนเป็นตัวใหญ่ ตัวอักษรขึ้นต้นคำสามารถเขียนเป็นตัวใหญ่ได้ แล้วแต่มาตรฐานการตั้งชื่อตัวแปรที่ใช้ อย่างไรก็ตามชื่อของเงื่อนไขบังคับสามารถเขียนเป็นตัวเล็กทั้งหมดก็ได้

อย่างไรก็ตาม ในกรณีที่เรามีความจำเป็นต้องสร้างหรือแก้ไขเงื่อนไขบังคับบูรณภาพใดๆ หลังจากที่ได้สร้างตารางนั้นๆ ไปแล้ว นอกจากนี้ยังอาจมีการเพิ่มข้อมูลในตารางแล้ว ทำให้ไม่มีความสะดวกในการลบตารางนั้นๆ ทิ้งเพื่อสร้างตารางใหม่พร้อมๆ กับเงื่อนไขบังคับบูรณภาพ เงื่อนไขบังคับสามารถสร้างได้ด้วยคำสั่ง ALTER TABLE ได้

ตัวอย่างที่ 7.12 การกำหนดเงื่อนไขบังคับคีย์หลัก (PRIMARY KEY CONSTRAINT)

การกำหนดเงื่อนไขบังคับคีย์นักศึกษาพร้อมกับ ALTER TABLE



จากตัวอย่าง คำสั่งที่ใช้ในการเพิ่มเงื่อนไขบังคับคือคำสั่งตั้งแต่ ALTER TABLE เป็นต้นไปเท่านั้น สำหรับคำสั่ง CREATE TABLE ในด้านบนนำมาแสดงเพื่อให้เห็นโครงสร้างของตารางแต่เพียงอย่างเดียว เราใช้การสร้างเงื่อนไขบังคับในลักษณะนี้กรณีที่ตารางข้อมูลถูกสร้างเรียบร้อยและอาจบรรจุข้อมูลในตารางแล้ว โดย

ALTER TABLE	คำสั่งที่ใช้ในการแก้ไขตาราง จำเป็นต้องระบุไว้เพื่อให้ระบบจัดการฐานข้อมูลแก้ไขตารางที่ระบุ สำหรับทำการเพิ่มเงื่อนไขบังคับที่ผูกติดกับตารางนั้นๆ
student	ชื่อตาราง คือตารางที่ต้องการเพิ่มเงื่อนไขบังคับ ในตัวอย่างคือตารางนักศึกษา
ADD CONSTRAINT	คำสั่งในการกำหนดเงื่อนไขบังคับ เป็นคำสั่งที่ต้องคงไว้สำหรับการสร้างเงื่อนไขบังคับทุกๆ ประเภท
PK_student	ชื่อของเงื่อนไขบังคับ ในที่นี้คือชื่อของเงื่อนไขบังคับคีย์หลักของตารางนักศึกษา

PRIMARY KEY ประเภทของเงื่อนไขบังคับ สามารถแทนด้วย NOT NULL, DEFAULT, UNIQUE KEY, FOREIGN KEY หรือ CHECK ขึ้นอยู่กับประเภทของเงื่อนไขบังคับที่ต้องการสร้าง

sid คอลัมน์ที่กำหนด เป็นคอลัมน์ที่เงื่อนไขบังคับที่สร้างขึ้นนั้นๆ ตรวจสอบค่า สำหรับเงื่อนไขบังคับบางประเภทอาจมีรูปแบบที่ซับซ้อนกว่านี้ขึ้นอยู่กับประเภทของเงื่อนไขบังคับที่สร้าง ซึ่งสามารถศึกษาได้จากตัวอย่างการสร้างเงื่อนไขบังคับในตัวอย่างต่อๆ ไป

รูปแบบการสร้างเงื่อนไขบังคับทั้ง 3 รูปแบบได้แก่ 1) การกำหนดไว้หลังคอลัมน์พร้อมๆ กับการสร้างตาราง 2) การกำหนดไว้ท้ายการสร้างตาราง และ 3) การสร้างเงื่อนไขบังคับภายหลังจากการสร้างตาราง เป็นรูปแบบที่สามารถนำมาใช้สร้างเงื่อนไขบังคับประเภทอื่นๆ ได้เช่นเดียวกัน ซึ่งการเลือกใช้รูปแบบใดนั้นขึ้นอยู่กับลักษณะการใช้งาน ความเหมาะสม และรายละเอียดปลีกย่อยที่อาจไม่ได้กล่าวถึงของการสร้างเงื่อนไขบังคับประเภทต่างๆ

ตัวอย่างต่อไปนี้แสดงการสร้างเงื่อนไขบังคับประเภทต่างๆ โดยเลือกใช้รูปแบบที่เหมาะสม

ตัวอย่างที่ 7.13 การสร้างคีย์หลักจากคีย์ประกอบ (MULTI-COLUMN PRIMARY KEY CONSTRAINT)

การกำหนดเงื่อนไขบังคับคีย์หลัก โดยกำหนดให้คอลัมน์ sid และ cid เป็นคีย์หลัก ในตาราง enrolled

```
CREATE TABLE enrolled
(
        cid     CHAR(13),
        sid     CHAR(8),
        grade   FLOAT
        CONSTRAINT PK_enrolled PRIMARY KEY (cid, sid)
);
```

จากตัวอย่างเราสามารถกำหนดให้คีย์หลักสร้างจากคอลัมน์มากกว่า 1 คอลัมน์ได้ ซึ่งเราเรียกว่า Composite Primary Key โดยการกำหนดรายการคอลัมน์ที่รวมกันเป็นคีย์หลักไว้ดังแสดงคือ PRIMARY KEY (cid, sid) ทำให้ระเบียน 2 ระเบียนหรือมากกว่า ไม่สามารถมีค่า cid และ sid ที่เหมือนกันทั้ง 2 คอลัมน์พร้อมกัน ในกรณีนี้เป็นการกำหนดไม่ให้นักศึกษาลงทะเบียนรายวิชาที่ตนเองลงทะเบียนไปแล้ว (เป็นเพียงตัวอย่างเท่านั้นอาจไม่ตรงตามหลักการลงทะเบียนทั่วไป) เราไม่สามารถสร้างเงื่อนไขบังคับนี้โดยใช้รูปแบบการเขียน PRIMARY KEY ไว้หลังคอลัมน์ cid และ sid ได้ในกรณีนี้เนื่องจาก PRIMARY KEY ต้องมีเพียงชุดเดียวเท่านั้น

ตัวอย่างที่ 7.14 การกำหนดเงื่อนไขบังคับห้ามเป็นค่าว่าง (NOT NULL CONSTRAINT)

การกำหนดเงื่อนไขบังคับค่าว่าง โดยกำหนดให้คอลัมน์ cid และ name ในตาราง student ห้ามเป็นค่าว่าง sid เป็นคีย์หลัก

```
CREATE TABLE student
(
        cid     CHAR(13) NOT NULL,
        sid     CHAR(8) PRIMARY KEY,
        name    VARCHAR(50) NOT NULL,
        login   VARCHAR(50),
        age     INTEGER,
        gpa     REAL
);
```

จากตัวอย่างเงื่อนไขบังคับแต่ละประเภทสามารถสร้างพร้อมๆ กันได้ PRIMARY KEY CONSTRAINT สามารถสร้างได้ด้วยรูปแบบการนำมาเขียนไว้ท้ายการสร้างตารางในขณะที่ NOT NULL CONSTRAINT นิยมเขียนในลักษณะนี้ เนื่องจากรายละเอียดปลีกย่อยของ CONSTRAINT ที่เกี่ยวข้องกับค่าโดยปริยาย ซึ่งไม่ได้กล่าวถึงในที่นี้

เงื่อนไขบังคับดังกล่าวทำให้การเพิ่มระเบียนใหม่ในตารางนักศึกษาห้ามปล่อยให้ค่า cid และ name รวมถึง sid เป็นค่าว่าง

ตัวอย่างที่ 7.15 การกำหนดเงื่อนไขบังคับอื่นๆ ได้แก่ DEFAULT, UNIQUE, CHECK AND OTHER CONSTRAINTS

การกำหนดเงื่อนไขบังคับต่างๆ

```
CREATE TABLE student
(
        cid     CHAR(13) UNIQUE KEY,
        sid     CHAR(8) PRIMARY KEY,
        name    VARCHAR(50) NOT NULL,
        login   VARCHAR(50),
        age     INTEGER DEFAULT 18,
        gpa     REAL,
        CONSTRAINT CK_student_gpa CHECK (gpa BETWEEN 0 AND 4)
);
```

จากตัวอย่างมีการกำหนดเงื่อนไขบังคับกับคอลัมน์แต่ละคอลัมน์ดังนี้

cid ห้ามไม่ให้ค่าในคอลัมน์นี้ซ้ำกันระหว่างระเบียน

sid เป็นคีย์หลัก

name ห้ามเป็นค่าว่าง

login ไม่ได้ระบุเงื่อนไขบังคับใดๆ สามรถเป็นค่าว่างได้กรณีไม่ระบุค่า แต่ยังคงต้องเป็นข้อมูลชนิด VARCHAR ความยาวไม่เกิน 50 เท่านั้น

age ค่าโดยปริยายกรณีไม่ระบุ จะมีค่าเท่ากับ 18

gpa ตรวจสอบค่าจะต้องอยู่ในช่วง 0.00 ถึง 4.00 เท่านั้น

รูปแบบการกำหนดเงื่อนไขบังคับสามารถเลือกได้ตามความเหมาะสม

ตัวอย่างที่ 7.16 การกำหนดเงื่อนไขบังคับคีย์ภายนอก (FOREIGN KEY CONSTRAINT)

การกำหนดเงื่อนไขบังคับคีย์ภายนอก กำหนดให้ค่ารหัสนักศึกษา sid ในตารางลงทะเบียน enrolled ต้องอ้างอิงถึงถึง sid ในตาราง student เท่านั้น

```
CREATE TABLE enrolled
(
        cid     CHAR(13) NOT NULL,
        sid     CHAR(8) NOT NULL,
        grade   FLOAT,
        CONSTRAINT FK_student_enrolled FOREIGN KEY (sid) REFERENCES student (sid)
);
```

จากตัวอย่างเงื่อนไขบังคับคีย์ภายนอกสร้างจาก **CONSTRAINT FK_student_enrolled FOREIGN KEY (sid) REFERENCES student (sid)** โดย

CONSTRAINT	เป็นคำสั่งที่ใช้ในการสร้างเงื่อนไขบังคับคีย์ภายนอก
FK_student_enrolled	เป็นชื่อของเงื่อนไขบังคับ
FOREIGN KEY	คือประเภทของเงื่อนไขบังคับ
(sid)	คือชื่อของคอลัมน์ในตารางปัจจุบันซึ่งค่าในตารางนี้อ้างอิงถึงตารางหลัก ข้อมูลที่จะบรรจุในคอลัมน์นี้ต้องปรากฏอยู่ในระเบียนใดๆ ของตารางหลักและคอลัมน์ที่กำหนด
REFERENCES	เป็นส่วนของคำสั่งสร้างเงื่อนไขบังคับคีย์ภายนอก วางไว้ระหว่างวัตถุอ้างอิงทั้ง 2
student (sid)	คือชื่อของตาราง (student) และชื่อของคอลัมน์ (sid) ในตารางหลัก

ในที่นี้การเพิ่มข้อมูลในตารางการลงทะเบียน รหัสนักศึกษาที่จะเพิ่มจะต้องปรากฏอยู่ในตาราง student ด้วย เพื่อป้องกันความผิดพลาดของการกรอกรหัสนักศึกษาผิดในตารางลงทะเบียน

ตัวอย่างที่ 7.17 การลบเงื่อนไขบังคับDROP CONSTRAINT

การลบเงื่อนไขบังคับ—CONSTRAINT คำสั่งต่อไปนี้แสดงการลบเงื่อนไขบังคับ PRIMARY KEY ในตาราง student ที่ได้สร้างไว้ก่อนหน้า

```
DROP CONSTRAINT PK_student;
```

เงื่อนไขบังคับ PK_student จะถูกลบทิ้ง คอลัมน์ sid จะไม่ได้รับการตรวจสอบค่าให้เป็นไปตามเงื่อนไขคีย์หลักอีกต่อไป กล่าวคือค่าในคอลัมน์ sid สามารถเป็นค่าว่างและอาจมีค่าซ้ำกันได้ภายในตาราง

เดียวกัน รูปแบบการลบเงื่อนไขบังคับดังกล่าวใช้กับเงื่อนบังคับทุกประเภท โดยเปลี่ยนเพียงชื่อของเงื่อนไขบังคับ PK_student เป็นเงื่อนไขบังคับที่ต้องการลบ

การกำหนดเงื่อนไขบังคับบูรณภาพ หรือ integrity constraints ต่างๆ เป็นการกำหนดนิยามของโครงสร้างของฐานข้อมูลให้มีความสามารถในการตรวจสอบการแก้ไขข้อมูลใดที่ข้อมูลในฐานข้อมูลต้องมีสภาพถูกต้องสอดคล้องกับการใช้งานจริงอยู่ตลอดเวลา นับเป็นประโยชน์สำคัญที่ได้รับจากการใช้ระบบจัดการฐานข้อมูลเชิงสัมพันธ์ SQL ในการสร้างเงื่อนไขบังคับบูรณภาพได้แก่คำสั่ง CREATE CONSTRAINT ชื่อเงื่อนไขบังคับบูรณภาพต่างๆ ตามข้อกำหนดของฐานข้อมูล และจากการออกแบบฐานข้อมูล สามารถรองรับได้ด้วย CONSTRAINT: PRIMARY KEY, NOT NULL, DEFAULT, UNIQUE KEY, FOREIGN KEY และ CHECK

SQL อนุญาตให้ผู้พัฒนาระบบฐานข้อมูลสร้างกลไกการรักษาบูรณภาพของข้อมูลที่ซับซ้อนมากกว่าการใช้ CONSTRAINT เช่นการใช้ TRIGGERS ซึ่งจะได้กล่าวถึงในบทที่ 8 สำหรับในหัวข้อถัดไปเป็นการอธิบายการสร้างดัชนี ซึ่งเป็นความสามารถอีกอันหนึ่งของระบบจัดการฐานข้อมูลเชิงสัมพันธ์ที่ SQL รองรับได้

7.6 ดัชนี (Index)

ดัชนี/อินเด็กซ์หรือ index เป็นกลไกในการเข้าถึงข้อมูลระเบียนใดๆ ได้รวดเร็วขึ้น เป็นความสามารถของ SQL ในการจัดการฐานข้อมูลระดับกายภาพเพื่อเพิ่มประสิทธิภาพของการใช้งานฐานข้อมูล หัวข้อต่อไปนี้อธิบายถึงหลักการและแนวคิดเกี่ยวกับ index ตลอดจนการจัดการ index ด้วยคำสั่ง SQL

7.6.1 แนวคิดทั่วไปเกี่ยวกับ Index

พิจารณาข้อมูลตารางนักศึกษาที่ได้อ้างถึงในบทนี้ ถ้าข้อคำถามที่ได้รับการใช้งานบ่อยๆ คือ “แสดงรายชื่อนักศึกษาเรียงตามลำดับตัวอักษรตามพจนานุกรม” ระบบจัดการฐานข้อมูลจะมีกลวิธีใดในการจัดเรียงข้อมูลเพื่อแสดงผลข้อมูลให้เรียงตามลำดับ หากข้อมูลในตารางของนักศึกษาในฐานข้อมูลนั้นไม่ได้เรียงตามลำดับอยู่ ซึ่งอาจจะเรียงตามรหัสนักศึกษาหรือไม่ก็ได้ เนื่องจากข้อมูลที่เพิ่มเข้ามาใหม่มักต่อท้ายข้อมูลเดิม ข้อมูลในตารางที่จัดเก็บในอุปกรณ์จัดเก็บจึงมักไม่ได้เรียงลำดับตามตัวอักษร

Student

cid	sid	name	login	age	gpa
3100904022132	B5075666	สมชาย	somchai@it	18	3.44
3100904032132	B5075688	สมศรี	somsri@com	18	3.21
3100905622132	B5075650	สมศรี	somsri@it	19	3.82
3100904055132	B5075831	สมศักดิ์	somsak@med	21	1.80
3100904078132	B5075832	สมน้ำหน้า	somnamna@math	22	2.00

name
สมชาย
สมศรี
สมศรี
สมศักดิ์
สมน้ำหน้า

ภาพที่ 7-2 ตารางข้อมูลนักศึกษาและรูปแบบข้อมูลที่ใช้บ่อย

จากตัวอย่างนี้ระบบจัดการฐานข้อมูลจึงต้องนำรายชื่อนักศึกษาที่ได้รับจากการประมวลผลการแสดงข้อมูลในรูปด้านขวา มาเรียงลำดับอีกทีหนึ่ง

การเรียงลำดับนั้นเป็นการประมวลผลที่ต้องใช้เวลาพอสมควร เพื่อให้เข้าใจถึงหลักการประมวลผลการเรียงลำดับข้อมูล เราสามารถพิจารณาวิธีการเรียงลำดับข้อมูลรายชื่อนักศึกษาด้วยวิธีการเรียงลำดับข้อมูลต่อไปนี้

รอบที่ 1	สมชาย	สมศรี	สมศรี	สมศักดิ์	สมน้ำหน้า
รอบที่ 2	สมชาย	สมศรี	สมศรี	สมศักดิ์	สมน้ำหน้า
อบที่ 3	สมชาย	สมศรี	สมศรี	สมศักดิ์	สมน้ำหน้า
รอบที่ 4	สมชาย	สมศรี	สมศรี	สมศักดิ์	สมน้ำหน้า
สิ้นสุด	สมชาย	สมน้ำหน้า	สมศรี	สมศรี	สมศักดิ์

name
สมชาย
สมน้ำหน้า
สมศรี
สมศรี
สมศักดิ์

ภาพที่ 7-3 ตัวอย่างขั้นตอนการเรียงลำดับข้อมูล

การเรียงลำดับที่ได้แสดงนั้นมีหลักการคร่าวๆ คือการเปรียบเทียบข้อมูลในลำดับแรกกับลำดับถัดไป และให้ทำการเรียงลำดับข้อมูลนั้น จากนั้นนำข้อมูลลำดับถัดไปมาเปรียบเทียบกับข้อมูลที่ได้เรียงลำดับไว้แล้วว่าข้อมูลใหม่ที่นำมาเปรียบเทียบควรอยู่ในตำแหน่งใดของรายการที่ได้เรียงไว้แล้ว จากนั้นนำข้อมูลตัวถัดไปมาเปรียบเทียบอีกไปเรื่อยๆ ซึ่งสามารถอธิบายได้ตามขั้นตอนที่แสดงด้านบนดังนี้

รอบที่ 1 พิจารณาข้อมูลรายชื่อนักศึกษาเริ่มต้นที่ไม่ได้เรียงกันตามลำดับตัวอักษร ในบรรทัด “รอบที่ 1” ซึ่งยังไม่ได้ทำการเรียงลำดับข้อมูลใดๆ ให้ทำการเปรียบเทียบข้อมูลในลำดับแรกกับลำดับถัดมาคือ “สมชาย” และ “สมศรี” ว่าควรจะเรียงลำดับตามตัวอักษรอย่างไร เราพบว่าข้อมูลเรียงตามลำดับตัวอักษรอยู่แล้วให้ทำการเปรียบเทียบในรอบถัดไป

รอบที่ 2 หมายถึงได้มีการเปรียบเทียบข้อมูลแล้วจำนวน 1 คู่ใน 2 ลำดับแรก จากนั้นทำการเปรียบเทียบในรอบที่ 2 โดยการนำรายชื่อในลำดับที่ 3 คือ “สมศรี” มาเปรียบเทียบกับรายการ “สมชาย สมศรี” ว่ารายชื่อที่นำมาเปรียบเทียบใหม่ควรอยู่ที่ใด โดยจะต้องทำการเปรียบเทียบข้อมูลตั้งแต่ตัวแรกคือเปรียบเทียบกับ “สมชาย” ไปจนกระทั่งพบตำแหน่งที่สามารถวางข้อมูลได้คือหลัง “สมศรี” คนแรก

รอบที่ 3 หลังจากนั้นให้นำรายชื่อถัดไปได้แก่ “สมศักดิ์” มาเปรียบเทียบกับรายการที่ได้เรียบลำดับแล้ว “สมชาย สมศรี สมศรี” เพื่อหาตำแหน่งว่างของ “สมศักดิ์” โดยต้องเริ่มต้นเปรียบเทียบกับ “สมชาย” ไปจนกระทั่งพบว่าต้องวางไว้หลัง “สมศรี” คนที่ 2

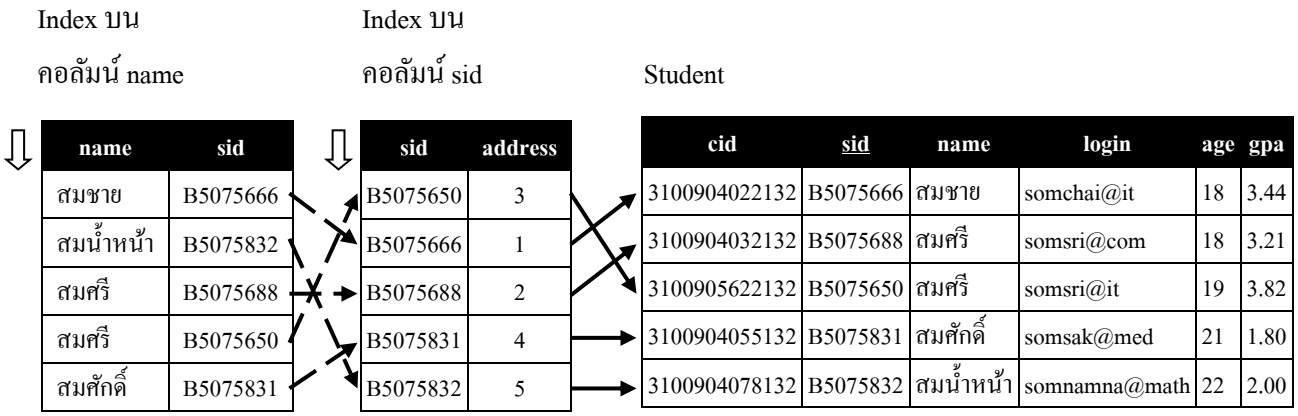
รอบที่ 4 หลังจากรอบที่ 3 คือรอบที่ 4 ซึ่งเป็นรอบสุดท้ายตามจำนวนข้อมูลที่มี นำ “สมน้ำหน้า” มาเปรียบเทียบกับรายการ “สมชาย สมศรี สมศรี สมศักดิ์” โดยต้องเริ่มเปรียบเทียบกับ “สมชาย” ไปจนกระทั่งพบว่าต้องวางไว้ระหว่าง “สมชาย” และ “สมศรี”

เราพบว่าการเรียงลำดับที่ได้ยกตัวอย่างนั้น การต้องมีการเปรียบเทียบข้อมูลเป็นจำนวนรอบเท่าๆ กับจำนวนข้อมูลที่มีอยู่ และแต่ละรอบก็จะต้องทำการเปรียบเทียบข้อมูลเท่าๆ กับจำนวนข้อมูลที่มีอยู่เช่นกัน ทำให้จะต้องมีการเปรียบเทียบค่าถึง จำนวนข้อมูลยกกำลังสอง หรือถ้าข้อมูลมีจำนวน n ระเบียน วิธีการเรียงลำดับนี้จะต้องทำการเปรียบเทียบข้อมูลเป็นจำนวนประมาณ n^2 ครั้ง ซึ่งในการใช้งานจริงนั้นข้อมูลนักศึกษาอาจมีมากถึงกว่า 10,000 ระเบียน เวลาที่ใช้ในการเปรียบเทียบข้อมูลนักศึกษา $10{,}000^2$ ครั้งที่ยังไม่รวมถึงเวลาในการอ่านข้อมูล การปรับลำดับของข้อมูลนั้น ใช้เวลาในการประมวลผลอย่างมาก

วิธีการเรียงลำดับข้อมูลที่ได้ยกตัวอย่างอาศัยอัลกอริธึมในการเรียงลำดับอย่างง่าย ถึงแม้ว่าเราจะใช้อัลกอริธึมในการเรียงลำดับที่มีประสิทธิภาพมากกว่านี้ เราก็สามารถทำได้ดีที่สุดที่ $n \log_2 n$ ซึ่งยังคงเป็นการประมวลผลที่ใช้เวลามากสำหรับของมูลขนาดใหญ่ หากระบบจัดการฐานข้อมูลไม่มีกลไกในการจัดการที่ดีพอนั้น ข้อคำถามบางคำถามที่ควรจะใช้เวลาไม่กี่วินาทีอาจใช้เวลาเป็นชั่วโมงในการประมวลผล และหากข้อคำถามในการแสดงข้อมูลแบบเรียงลำดับนี้จะต้องกระทำบ่อยครั้งในและวันหรือในแต่ละชั่วโมงหรือแม้แต่นาที ระบบจัดการฐานข้อมูลอาจมีประสิทธิภาพลดลงอีกเนื่องจากต้องยุ่งอยู่กับการประมวลผลใหม่ตลอดเวลา

ปัญหาการใช้ความมานะอย่างมากในการเรียงลำดับข้อมูลนี้เป็นเพียงตัวอย่างหนึ่งของความจำเป็นในการที่ระบบจัดการฐานข้อมูลต้องมีกลไกในการแก้ปัญหาการจัดเก็บของข้อมูลให้สอดคล้องกับการใช้งาน ในความเป็นจริงแล้วยังมีปัญหาที่สำคัญอื่นๆ อีก เช่นการระบุตำแหน่งทางกายภาพบนอุปกรณ์จัดเก็บข้อมูลของระบบจัดการฐานข้อมูลว่าข้อมูลระเบียนใดๆ อยู่ที่ตำแหน่งใดของอุปกรณ์จัดเก็บในการเข้าถึงได้รวดเร็ว

Index เป็นกลไกที่ใช้ช่วยให้การเข้าถึงข้อมูลในลักษณะที่ได้กล่าวมารวดเร็วขึ้น โดยมีหลักการทำงานง่ายๆ คือให้ทำการสร้างข้อมูลที่เรียงลำดับไว้อยู่แล้วแยกต่างหากจากตาราง เช่น ให้สร้าง index ของคอลัมน์ชื่อนักศึกษา และ index ของคอลัมน์รหัสนักศึกษา



ภาพที่ 7-4 หลักการทำงานของการแสดงข้อมูลด้วย index

พิจารณาการสร้างผลลัพธ์จากข้อคำถามต่อไปนี้ โดยอาศัย index

"แสดงรายการข้อมูลนักศึกษาประกอบด้วยรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยเรียงตามลำดับรหัสนักศึกษา" การสร้างรายการข้อมูลนักศึกษาดังกล่าวเริ่มจากตาราง index บนคอลัมน์ sid ซึ่งประกอบด้วยรหัสนักศึกษา sid และตำแหน่งที่อยู่ของระเบียนเชิงตรรกะ ดังแสดงในตารางข้อมูล กระบวนการแสดงผลเริ่มจากการเข้าถึงข้อมูลแรกได้แก่ นักศึกษารหัส "B5075650" ซึ่งอยู่ในตำแหน่งที่ 3 จึงได้ผลลัพธ์เป็นระเบียนแรก คือ

นักศึกษา “สมศรี” จากนั้นพิจารณานักศึกษาคนถัดมาใน index ได้แก่นักศึกษา “B5075666” ซึ่งอยู่ในลำดับที่ 1 ของตารางข้อมูล (ดังแสดงตามแผนภาพด้านบนด้วยเส้นทึบ) เมื่อดำเนินการอย่างนี้เรื่อยไปจนได้ผลลัพธ์ตามข้อคำถามซึ่งแสดงได้ดังนี้

sid	name	gpa
B5075650	สมศรี	3.82

sid	name	gpa
B5075650	สมศรี	3.82
B5075666	สมชาย	3.44

sid	name	gpa
B5075650	สมศรี	3.82
B5075666	สมชาย	3.44
B5075688	สมศรี	3.21
B5075831	สมศักดิ์	1.80
B5075832	สมน้ำหน้า	2.00

ภาพที่ 7-5 ผลลัพธ์การสร้างตารางข้อมูลจาก index ในภาพที่ 7-4

การกระทำในลักษณะดังกล่าวเปรียบเสมือนกับดัชนีในหนังสือซึ่งแสดงรายการของคำสำคัญตามลำดับตัวอักษร ควบคู่กับหน้าของหนังสือที่คำนั้นๆ ปรากฏ เช่นเดียวกับการสร้าง index ในระบบจัดการฐานข้อมูลดังได้แสดงนั่นเอง ที่ข้อมูล index เรียงลำดับในคอลัมน์ใดๆ ควบคู่กับตำแหน่งที่อยู่ในตารางข้อมูลจริง ซึ่งกลไกดังกล่าวจะช่วยให้การเข้าถึงข้อมูลนั่นรวดเร็วขึ้นมาก โดยเฉพาะการจัดการกับข้อมูลจำนวนมหาศาล

พิจารณาการสร้างผลลัพธ์จากข้อคำถามเพิ่มเติมต่อไปนี้ โดยอาศัย index

“แสดงรายการข้อมูลนักศึกษาประกอบด้วยรหัสนักศึกษา ชื่อ และเกรดเฉลี่ยเรียงตามลำดับ**ชื่อนักศึกษา**”

sid	name	gpa
B5075666	สมชาย	3.44

sid	name	gpa
B5075650	สมศรี	3.82
B5075832	สมน้ำหน้า	2.00

sid	name	gpa
B5075666	สมชาย	3.44
B5075832	สมน้ำหน้า	2.00
B5075688	สมศรี	3.21
B5075650	สมศรี	3.82
B5075831	สมศักดิ์	1.80

ภาพที่ 7-6 ผลลัพธ์การสร้างตารางข้อมูลจาก index ในภาพที่ 7-4

จากภาพ ให้ใช้ภาพ index ประกอบการอธิบาย โดยการแสดงข้อมูลเรียงลำดับตามตัวอักษรของชื่อนั้นเริ่มจากตาราง index ของ name นักศึกษา “สมชาย” เป็นนักศึกษาคนแรก ซึ่งมีรหัสนักศึกษา “B5075666” เราสามารถค้นหาตำแหน่งที่อยู่ของระเบียนในตารางข้อมูลได้อย่างรวดเร็วเนื่องจากข้อมูลใน index บนคอลัมน์ sid เรียงลำดับไว้เช่นกัน (ดังแสดงตามเส้นประ) จากนั้นระบบจัดการฐานข้อมูลจะสามารถเข้าถึงระเบียนข้อมูลได้ทันทีด้วยตำแหน่งของระเบียนใน index บนคอลัมน์ sid ดังเช่นตัวอย่างที่แล้ว เมื่อดำเนินการอย่างนี้เรื่อยไปจะได้ผลลัพธ์ที่เรียงตามรายชื่อ การดำเนินการเช่นนี้รวดเร็วกว่าการต้องเรียงลำดับทุกครั้งในการแสดงข้อมูลเป็นอย่างมาก

ขอให้พิจารณาว่า การแสดงข้อมูลตามข้อคำถามด้วย index ทั้ง 2 กรณีต่างกัน กล่าวคือ สำหรับ sid นั้น ข้อมูลใน index ชี้ไปยังตำแหน่งในตารางข้อมูล แต่ index บนคอลัมน์ name ข้อมูลชี้ไปยัง index บนคอลัมน์ sid อีกทีหนึ่ง ทั้งนี้เนื่องจากข้อมูลที่บรรจุใหม่อาจะมีการเปลี่ยนตำแหน่งในตารางได้ตลอดเวลา ดังนั้นการสร้าง index ที่อ้างอิงกับตำแหน่งที่อยู่ในตารางข้อมูลเพียงหนึ่งชุดจะทำให้ระบบจัดการฐานข้อมูลมีประสิทธิภาพมากกว่า กล่าวคือ ระบบจัดการฐานข้อมูลไม่จำเป็นต้องปรับปรุง index ทุกๆ index ที่สร้างขึ้น เพียงแต่ปรับปรุง index หลักที่อ้างอิงถึงตำแหน่งที่อยู่ของระเบียนเท่านั้น ซึ่งโดยทั่วไปแล้วระบบจัดการฐานข้อมูลจะทำการจัดเก็บข้อมูลโดยอาศัยกลไก index บนคอลัมน์ที่เป็น PRIMARY KEY นั่นเอง การที่การดำเนินการต่างๆ ในตารางจำเป็นต้องอาศัย PRIMARY KEY นี้เอง การเลือก PRIMARY KEY ในกรณีที่มี candidate key จึงต้องคำนึงถึงความกระชับของ PRIMARY KEY ด้วย จากตัวอย่างนี้เราสามารถสร้าง index ได้บนคอลัมน์ใดๆ ก็ได้ที่ต้องการเพื่อเพิ่มประสิทธิภาพการแสดงผลข้อมูล โดย index ที่สร้างขึ้นจะอ้างอิงกับ PRIMARY KEY

อย่างไรก็ตามการสร้าง index ไม่ได้มีแต่ข้อดีเสมอไป เนื่องจาก index ที่สร้างขึ้นจะต้องถูกปรับปรุงให้ถูกต้องอยู่เสมอ เช่นเมื่อมีการเพิ่มข้อมูลนักศึกษาใหม่ index บนคอลัมน์รายชื่อนักศึกษาก็จะต้องมีการจัดการใหม่เพื่อให้เรียงลำดับตามรายชื่อนักศึกษาเช่นเดิม ซึ่งก็เป็นการใช้ทรัพยากรของระบบที่รองรับระบบจัดการฐานข้อมูลส่วนหนึ่ง ถ้าเราสร้าง index ที่เกินความจำเป็น เช่นสร้าง index บนคอลัมน์ gpa หรือ age เพิ่มขึ้นอีก ทุกครั้งที่มีการเพิ่มหรือแก้ไขระเบียนนักศึกษา ระบบจัดการฐานข้อมูลตัวปรับปรุง index ของ gpa และ age เพิ่มขึ้นอีก เป็นการเพิ่มภาระให้กับระบบ นอกจากนี้ข้อคำถามที่มีการเข้าถึงข้อมูลโดยอาศัยกลไก index บนคอลัมน์ gpa และ age อาจมีไม่บ่อยครั้ง จึงเป็นการไม่คุ้มค่าที่จะสร้าง index โดยไม่จำเป็น ทั้งนี้นักวิเคราะห์ระบบ—SA และผู้ดูแลระบบ—DBA มีหน้าที่วิเคราะห์การสร้าง index ที่เหมาะสมตามข้อกำหนดการใช้งานหรือจากสถิติการใช้งานจริง

ในทางปฏิบัติแล้ว ระบบจัดการฐานข้อมูลจะใช้เทคนิคที่มีประสิทธิภาพในการจัดการกับ index มากกว่านี้ โดยไม่ได้ทำ index ทุกระเบียน แต่จะอาศัยกลุ่มของระเบียนที่รวมกันเรียกว่า page โดยในแต่ละ page จะมีระเบียนของข้อมูลที่อาจเรียงกันตาม index หลัก ซึ่งมักจะเรียงกันตาม PRIMARY KEY เมื่อจะทำการเข้าถึงข้อมูลระเบียนใดๆ ระบบจัดการฐานข้อมูลจะค้นหาว่าระเบียนนั้นๆ อยู่ใน page ใด แล้วทำการเข้าถึงข้อมูลที่ page นั้นเพื่อเข้าถึงข้อมูลในระเบียนที่ต้องการเข้าถึงอีกทีหนึ่ง นอกจากนี้ยังมีเทคนิคและรายละเอียดปลีกย่อยอื่นๆ ของการจัดการฐานข้อมูลในระดับกายภาพซึ่งไม่ได้ลงรายละเอียดในเอกสารเล่มนี้

SQL รองรับแนวคิด index นี้ โดยอำนวยความสะดวกให้ผู้จัดการระบบฐานข้อมูลสร้างและจัดการ index ได้ด้วยคำสั่งที่ไม่ซับซ้อนเพื่อให้ระบบจัดการฐานข้อมูลอาศัยโครงสร้าง index ที่ได้นิยามไว้นี้ในการจัดการกับข้อมูล

7.6.2 การสร้าง Index

การสร้าง index ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

CREATE INDEX <index_name> ON <table_name> (<column_name>);

โดย

CREATE INDEX	คือคำสั่งที่ใช้ในการสร้างดัชนี
index_name	ชื่อของ index เพื่อใช้ในการจัดการในภายหลัง
ON	ประกอบคำสั่ง CREATE INDEX เพื่อระบุว่าสร้าง index “บน” คอลัมน์
table_name	ชื่อตารางที่ต้องการสร้าง index
column_name	ชื่อของคอลัมน์สำหรับสร้าง index

สามารถเขียนรูปแบบคำสั่งสร้าง index ที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

สร้าง ดัชนี <ชื่อดัชนี> บน <ชื่อตาราง>(<ชื่อคอลัมน์>);

ตัวอย่างที่ 7.18 การสร้าง index (CREATE INDEX)

การสร้าง index บนคอลัมน์ name ในตาราง student

CREATE INDEX idx_student_name ON student(name);

จากตัวอย่าง idx_student_name เป็นชื่อของ index ซึ่งเราจะนำมาใช้จัดการในภายหลัง คำสั่งดังกล่าวสร้างโครงสร้าง index ที่ระบุไว้ การเข้าถึงข้อมูลที่อ้างถึงชื่อนักศึกษาและอาศัยกลไกการทำงานของ index จะมีความรวดเร็วขึ้น

7.6.3 การลบ Index

การสร้าง index ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

DROP INDEX <index_name>;

โดย

DROP INDEX	คือคำสั่งที่ใช้ในการลบดัชนี
index_name	ชื่อของ index ที่ต้องการลบ

สามารถเขียนรูปแบบคำสั่งลบ index ที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

ลบ ดัชนี <ชื่อดัชนี>;

ตัวอย่างที่ 7.19 การลบ index (DROP INDEX)

การลบ index บนคอลัมน์ name ในตาราง student ที่ได้สร้างไว้

DROP INDEX idx_student_name;

จากตัวอย่าง idx_student_name เป็นชื่อของ index ซึ่งเราจะนำมาใช้จัดการในภายหลัง คำสั่งดังกล่าวสร้างโครงสร้าง index ที่ระบุไว้ การเข้าถึงข้อมูลที่อ้างถึงชื่อนักศึกษาและอาศัยกลไกการทำงานของ index จะมีความรวดเร็วขึ้น

7.7 วิว View

วิว เป็นตารางข้อมูลเสมือนที่ไม่ได้มีข้อมูลอยู่จริง สามารถเปรียบได้กับหน้ากากที่ครอบตารางความสัมพันธ์ในระบบจัดการฐานข้อมูลเชิงสัมพันธ์ เพื่อให้ได้ตารางของข้อมูลที่ตรงกับการใช้งาน เป็นการกำหนดสคีมาของฐานข้อมูลระดับภายนอก สำหรับประโยชน์และการจัดการวิวจะได้อธิบายในหัวข้อนี้

7.7.1 แนวคิดทั่วไปเกี่ยวกับ View

วิวมีประโยชน์หลักๆ 3 ประการ ได้แก่การสร้างตารางเสมือนเพื่อเพิ่มความสะดวกในการเรียกใช้งานข้อมูล การสร้างตารางเสมือนเพื่อคงความเป็นอิสระของข้อมูล และเรื่องเกี่ยวกับความปลอดภัยของข้อมูล

ในหัวข้อที่ 1.6.3 ได้กล่าวถึงการใช้งานวิวเพื่อสร้างตารางเสมือน ซึ่งเป็นการใช้งานประโยชน์หลักทั้ง 3 ประการของวิว โดยเฉพาะในเรื่องการคงความเป็นอิสระของข้อมูล นอกจากตัวอย่างในหัวข้อที่ 1.6.3 แล้ว ในที่นี้เราจะพิจารณาการใช้งานวิวด้วยกรณีตัวอย่างข้อมูลของตาราง 3 ตารางดังนี้

Faculty

fid	fname	sal
001	แพนเค้ก	40,000
007	สถิตย์	50,000
069	พอลล่า	30,000

Teaches

fid	cid
001	204205
007	204203
007	204204
007	204329
069	202102
069	202103

Course

cid	cname	credits
202102	Information Technology I	3
202103	Information Technology II	3
204203	Software Design and Development	4
204204	Database Design and Development	4
204205	Multimedia Design and Development	4
204329	Web Technology	3

ในกรณีที่ข้อมูลที่จะต้องใช้บ่อยๆ คือการแสดงรายชื่อของอาจารย์ผู้สอนกับรายวิชาที่สอน เราสามารถสร้างวิวที่แสดงข้อมูลจากทั้ง 3 ตารางได้ดังนี้

Faculty_Teaches_Course

fid	fname	cid	cname
001	แพนเค้ก	204205	Multimedia Design and Development
007	สถิตย์	204203	Software Design and Development
007	สถิตย์	204204	Database Design and Development
007	สถิตย์	204329	Web Technology
069	พอลล่า	202102	Information Technology I
069	พอลล่า	202103	Information Technology II

ซึ่งตารางข้อมูลนี้สามารถสร้างให้อยู่ในรูปของตารางเสมือนเพื่อเรียกใช้งานได้บ่อยครั้ง โดยทำการสร้างวิว faculty_teaces_course ขึ้นมาเพื่อความสะดวกได้

นอกจากนี้ข้อมูลเงินเดือน sal ของอาจารย์ยังเป็นข้อมูลที่เป็นความลับกับบุคคลทั่วไป แต่เนื่องจากฐานข้อมูลของมหาวิทยาลัยนี้จะต้องใช้ร่วมกันทั้งมหาวิทยาลัย เราสามารถสร้างวิว faculty_public ซึ่งไม่มีข้อมูลเงินเดือนของคณาจารย์เป็นหน้ากากครอบตาราง faculty ไว้ เพื่อให้**ระบบลงทะเบียน**สามารถเข้าถึงได้เฉพาะวิว faculty_public ในขณะที่**ระบบเงินเดือน**ของมหาวิทยาลัยสามารถเข้าถึงตาราง faculty ได้เพื่อจัดการเกี่ยวกับเงินเดือนของคณาจารย์

Faculty_Public

fid	fname
001	แพนเค้ก
007	สถิตย์
069	พอลล่า

7.7.2 การสร้าง View

การสร้าง view ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

CREATE VIEW <view_name> AS <select_statement>;

โดย

CREATE VIEW คือคำสั่งที่ใช้ในการสร้างวิว

view_name ชื่อของวิว เพื่อใช้ในการจัดการในภายหลัง

AS ประกอบคำสั่ง CREATE VIEW เพื่อระบุว่าสร้าง VIEW "ด้วย" คำสั่ง SELECT ที่กำหนด

select_statement รายการคำสั่ง SELECT ของ SQL ที่ใช้ในการสร้างวิว

สามารถเขียนรูปแบบคำสั่งสร้างวิวที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

สร้าง วิว <ชื่อวิว> บน <คำสั่ง SELECT>;

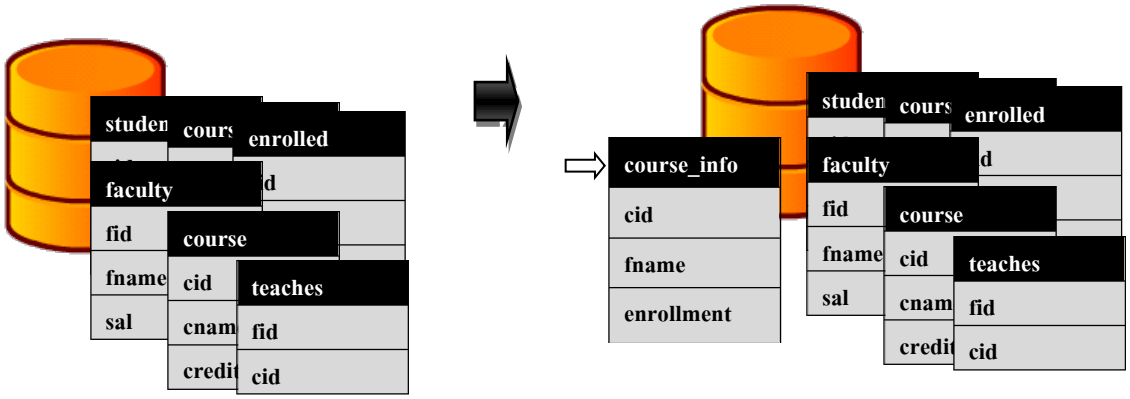
เนื่องจากวิวคือตารางเสมือน การสร้างวิวจึงใช้คำสั่ง SELECT ในการเลือกข้อมูลเพื่อสร้างตารางเสมือนจากตารางข้อมูลที่มีอยู่แล้ว เราเรียกวิวว่าตารางเหมือน—virtual table และตารางข้อมูลจริงในฐานข้อมูลว่าตารางหลัก—base table วัตถุที่เกิดขึ้นจากการสร้างวิวเป็นเพียงโครงสร้างของตารางเสมือน ในการใช้งานวิวนั้นเราใช้คำสั่ง SQL ในการแสดงข้อมูล แก้ไขข้อมูล และลบข้อมูลผ่านทางวิวเสมือนเป็นการกระทำกับตารางข้อมูลจริงตารางหนึ่งโดยระบบจัดการฐานข้อมูลมีกลไกในการจัดการข้อมูลให้มีผลต่อตารางหลักผ่านทางวิว ซึ่งเป็นกฎข้อหนึ่งในกฎ 12 ข้อของ E.F.Codd นั่นเอง สำหรับคำสั่ง SELECT นั้นเป็นคำสั่งในกลุ่มคำสั่งที่ใช้ในการจัดการข้อมูล—DML ของ SQL ที่จะได้อธิบายในบทถัดไป ในที่นี้จึงเป็นการศึกษาถึงรูปแบบการใช้คำสั่งในการสร้างวิวเท่านั้น

ตัวอย่างที่ 7.20 การสร้างวิว (CREATE VIEW)

สร้างวิว Course_info (*cid:* string, *fname:* string, *enrollment:* integer) ซึ่งประกอบด้วยคอลัมน์รหัสวิชา cid ชื่ออาจารย์ fname และจำนวนผู้ลงทะเบียน enrollment ซึ่งจะต้องใช้ข้อมูลจากตาราง teaches, faculty และ enrolled ประกอบกัน เพื่อแสดงรายการการลงทะเบียนของแต่ละรายวิชา โดยแสดงรหัสวิชา ชื่ออาจารย์ผู้สอน และจำนวนนักศึกษาที่ลงทะเบียน

```
CREATE VIEW course_info
AS
SELECT e.cid, fname, COUNT(*) AS enrollment
FROM teaches t, faculty t, enrolled e
WHERE t.fid = f.fid
AND t.cid = e.cid
GROUP BY e.cid;
```

จากตัวอย่าง ผลลัพธ์ที่ได้จากกาดำเนินการตามคำสั่ง SQL สร้างวิว คือการกำหนดโครงสร้างของวิวว่าประกอบด้วยคอลัมน์อะไรบ้าง และข้อมูลของแต่ละคอลัมน์ นำมาจากตารางใด เสมือนการสร้างสคีมาของตารางโดยปกติทั่วไปดังแผนภาพต่อไปนี้



คำสั่งสร้างวิวนั้นไม่ได้แสดงข้อมูลใดๆ การเรียกใช้วิวนั้นเราดำเนินการด้วยการใช้คำสั่ง SQL ดังเช่นที่กระทำกับตารางโดยทั่วไป ในกรณีนี้คือตาราง course_info ซึ่งเป็นวิวที่ได้สร้างขึ้น ยกตัวอย่างการแสดงข้อมูลทั้งหมดจากวิวด้วย SQL ต่อไปนี้

```
SELECT * FROM course_info ORDER BY cid
```

ได้ผลลัพธ์ดังตารางต่อไปนี้

Course_info

cid	fname	enrollment
204203	สถิตย์	1
204204	สถิตย์	2
204205	แพนเค้ก	1
204329	สถิตย์	2

ตัวอย่างการสร้างวิว Course_info นี้ นอกจากจะใช้ประโยชน์ในการใช้วิวเพื่อเพิ่มความสะดวกในการเรียกใช้งานข้อมูลที่ใช้บ่อยๆ ผ่านทางวิวที่สร้างขึ้นจาก SQL ที่มีความซับซ้อนแล้ว วิวดังกล่าวยังคงความเป็นอิสระของข้อมูลจากฐานข้อมูลระดับตรรกะ และป้องกันข้อผิดพลาดที่อาจจะเกิดขึ้นจากความซ้ำซ้อนของข้อมูลคอลัมน์ enrollment ซึ่งสามารถศึกษารายละเอียดได้ในบทที่ 1 หัวข้อที่ 1.6.3 ความเป็นอิสระของข้อมูล

ตัวอย่างที่ 7.21 การสร้างวิว (CREATE VIEW)

สร้างวิว Faculty_Teaches_Course (*fid:* string, *fname:* string, *cid:* string, *cname:* string)

```
CREATE VIEW faculty_teaches_course
AS
SELECT f.fid, fname, c.cid, cname
FROM faculty f, teaches t, course c
WHERE f.fid = t.tid AND t.cid = c.cid;
```

จากตัวอย่าง วิวดังกล่าวประกอบด้วยคอลัมน์ fid, fname, cid และ cname ซึ่งเป็นรายการข้อมูลที่เรียกใช้บ่อยครั้งว่าอาจารย์ท่านใด สอนรายวิชาใดบ้าง โดยข้อมูลสร้างจากตาราง faculty, teaches และ course เราสามารถยกตัวอย่างการใช้วิวดังกล่าวโดยการแสดงข้อมูลทั้งหมดด้วยคำสั่ง

```
SELECT * FROM faculty_teaches_course
```

ซึ่งผลลัพธ์แสดงได้ตามตาราง Faculty_Teaches_Course ในหน้าแรกของหัวข้อ 7.7.1

ตัวอย่างที่ 7.22 การสร้างวิว (CREATE VIEW)

สร้างวิว Faculty_Public (*fid:* string, *fname:* string)

```
CREATE VIEW faculty_public
AS
SELECT fid, fname
FROM faculty;
```

จากตัวอย่าง วิวดังกล่าวประกอบด้วยคอลัมน์รหัสอาจารย์ fid และชื่ออาจารย์ fname เท่านั้น ไม่มีคอลัมน์เงินเดือน sal ของอาจารย์ การพัฒนาระบบลงทะเบียนสามารถอ้างอิงถึงเฉพาะวิว faculty_public นี้ได้ โดยเก็บตารางหลัก faculty ที่มีข้อมูลเงินเดือนสำหรับระบบสารสนเทศเงินเดือนของมหาวิทยาลัยเท่านั้นที่สามารถเข้าถึงได้ การเรียกใช้ข้อมูลสามารถยกตัวอย่างได้ด้วยคำสั่ง

```
SELECT * FROM faculty_public
```

ซึ่งผลลัพธ์แสดงได้ตามตาราง Faculty_Public ในหน้าที่ 2 ของหัวข้อ 7.7.1

วิวต่างๆ ที่ได้สร้างขึ้นมานั้นสามารถรองรับคำสั่ง SQL ที่ใช้ในการแก้ไขและลบข้อมูลอีกด้วย สำหรับคำสั่งที่ใช้ในการแก้ไขและลบข้อมูลได้อธิบายไว้ในบทถัดไป

7.7.3 การลบ View

การลบ view ใช้คำสั่งที่มีรูปแบบอย่างง่ายดังนี้

DROP VIEW <view_name>;

โดย

DROP VIEW คือคำสั่งที่ใช้ในการลบวิว

view_name ชื่อของวิวที่ต้องการลบ

สามารถเขียนรูปแบบคำสั่งลบวิวที่สามารถเข้าใจได้ง่ายขึ้นเทียบกับภาษาไทยได้ดังนี้

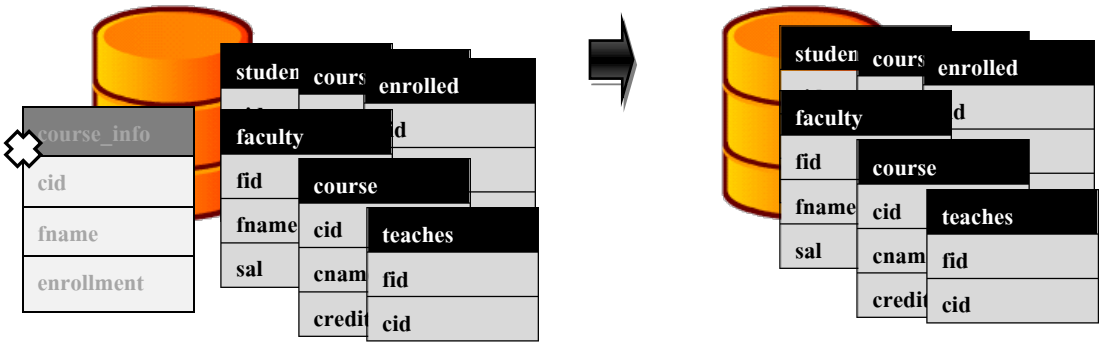
ลบ วิว <ชื่อวิว>;

ตัวอย่างที่ 7.23 การลบวิว (DROP VIEW)

การลบวิว Course_info ที่ได้สร้างไว้

DROP VIEW course_info;

การลบวิวเปรียบเสมือนการลบสคีมาของตารางโดยปกติทั่วไปดังแผนภาพต่อไปนี้



การลบวิวนั้นต่างจากการลบตารางเนื่องจากการลบวิวเป็นเพียงการลบสคีมาหรือโครงสร้างของตารางรางเสมือนเท่านั้น ข้อมูลที่บรรจุอยู่ในตารางหลักที่เกี่ยวข้องกับวิวยังคงอยู่ไม่ได้ถูกลบไปด้วย

7.8 บทสรุป

ภาษาสอบถามแบบมีโครงสร้าง SQL (Structured Query Language) เป็นภาษาหลักที่นิยมใช้ในการใช้งานระบบจัดการฐานข้อมูลเชิงสัมพันธ์ ซึ่งสามารถแบ่งเป็นชนิดตามลักษณะการใช้งานของคำสั่งได้แก่ ภาษานิยามข้อมูล ภาษาจัดการข้อมูล และภาษาควบคุม

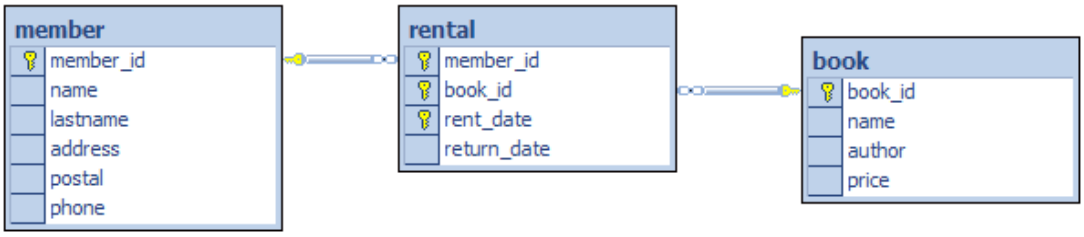
ภาษานิยามข้อมูลเป็นเพียงคำสั่งที่ใช้ในการกำหนดสคีมาของตารางข้อมูล ยังไม่ได้บรรจุหรือจัดการกับข้อมูลที่จะจัดเก็บ คำสั่งในกลุ่มที่ใช้ในการนิยามข้อมูลได้แก่ CREATE และ DROP DATABASE สำหรับสร้างและลบฐานข้อมูล CREATE, ALTER และ DROP TABLE สำหรับสร้าง เปลี่ยนแปลงโครงสร้าง และลบตารางตามลำดับ CREATE และ DROP VIEW ใช้ในการสร้างและลบวิว โดยวิวนั้นเปรียบเสมือนมุมมองของผู้ใช้ซึ่งครอบตารางข้อมูลที่บรรจุข้อมูลจริงอีกทีหนึ่ง และ CREATE และ DROP INDEX ใช้สำหรับสร้างและลบ index ซึ่งเป็นโครงสร้างที่เพิ่มความเร็วในการเข้าถึงข้อมูล

หลังจากที่เราได้สร้างโครงสร้างของข้อมูลซึ่งเป็นขั้นเริ่มต้นของการทำให้เกิดฐานข้อมูลแล้ว เราจะได้ใช้คำสั่ง SQL สำหรับเพิ่ม แก้ไข และลบข้อมูลต่อไป

7.9 แบบฝึกหัดท้ายบท

1. ให้เขียนคำสั่ง SQL สำหรับสร้างฐานข้อมูล library จาก ERD และสคีมาดังนี้

แผนภาพความสัมพันธ์ (Relationship Diagram)



สคีมา (Schema)

member เป็นตารางสำหรับจัดเก็บข้อมูลของสมาชิก

Field Name	Data Type	Description	Field Size	Required
member_id	varchar	รหัสประจำตัวสมาชิก ใช้เป็น PK	5	Yes
name	varchar	ชื่อลูกค้า	20	Yes
lastname	varchar	นามสกุลลูกค้า	20	Yes
address	varchar	ที่อยู่ลูกค้า	100	
postal	varchar	รหัสไปรษณีย์	5	
phone	varchar	เบอร์โทรศัพท์ลูกค้า	11	

book เป็นตารางจัดเก็บรายละเอียดของหนังสือ

Field Name	Data Type	Description	Field Size	Required
book_id	varchar	รหัสประจำเล่มหนังสือ ใช้เป็น PK	5	Yes
name	varchar	ชื่อหนังสือ	50	Yes
author	varchar	ผู้แต่ง	50	Yes
price	Money	ราคาหนังสือ		

rental เป็นตารางบันทึกการยืมคืน โดยบันทึกรหัสสมาชิกและรหัสหนังสือสำหรับการยืม พร้อมทั้งวันที่ยืมในการยืมแต่ละครั้งโดยยังไม่ใส่วันที่คืน เมื่อสมาชิกนำหนังสือมาคืนแล้วจึงทำการ UPDATE record นั้นๆ โดยใส่วันที่ที่สมาชิกนำหนังสือมาคืนในคอลัมน์ return_date

Field Name	Data Type	Description	Field Size	Required
member_id	varchar	รหัสของสมาชิกที่ยืมหนังสือ ใช้เป็น PK	5	Yes
book_id	varchar	รหัสของหนังสือที่ถูกยืม	5	Yes
rent_date	Date/Time	วันที่ยืมหนังสือ		Yes
return_date	Date/Time	วันที่คืนหนังสือ		

2. ให้เขียนคำสั่ง SQL สำหรับสร้างตารางความสัมพันธ์และเงื่อนไขบังคับของฐานข้อมูลสำหรับตัวอย่างร้านมือถือออนไลน์ 7-Elephant ในกรณีศึกษาของบทที่ 3 ให้กำหนดโดเมนของข้อมูลด้วยตนเอง
3. ให้เขียนคำสั่ง SQL สำหรับสร้างตารางความสัมพันธ์และเงื่อนไขบังคับของฐานข้อมูลสำหรับตัวอย่างมหาวิทยาลัยเทคโนโลยีสุรบุรุษ ในกรณีศึกษาของบทที่ 3 ให้กำหนดโดเมนของข้อมูลด้วยตนเอง
4. ให้เขียนคำสั่ง SQL สำหรับสร้างตารางความสัมพันธ์และเงื่อนไขบังคับของฐานข้อมูลสำหรับตัวอย่าง อู่เฉิดฉาย ในกรณีศึกษาของบทที่ 3 ให้กำหนดโดเมนของข้อมูลด้วยตนเอง
5. ให้เขียนคำสั่ง SQL สำหรับสร้างตาราง กำหนดเงื่อนไขบูรณภาพ ตลอดจนสร้างอินเด็กซ์และวิวของโครงงาน “เช่า-ฉัน-สิ”

บรรณานุกรมและเอกสารอ้างอิงประจำบท

ชนวัฒน์ ศรีสอ้าน. (2542). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

พงษ์พันธ์ ศิวิลัย. (2549). **SQL Server 2005 ฉบับสมบูรณ์.** กรุงเทพฯ : ซีเอ็ดยูเคชั่น.

ศุภกฤษฎิ์ นิวัฒนากูล. (2547). **เอกสารประกอบการบรรยายรายวิชา 204204 การออกแบบและพัฒนาฐานข้อมูล.** นครราชสีมา : มหาวิทยาลัยเทคโนโลยีสุรนารี.

Connolly, T. and Begg, C. (2004). **Database Systems: A Practical Approach to Design, Implementation and Management, 4th ed.** Boston : Addison-Wesley.

Elmarsi, R. and Navathe, S. B. (2006). **Fundamentals of Database Systems, 5th ed.** Boston : Addison-Wesley.

Ramakrishnan, R. and Gehrke, J. (2003). **Database Management Systems, 3rd ed**. Singapore : McGraw-Hill.